Question Bank with Complete Answers

Distributed Systems & Cloud Computing (CIE-407T)

Unit-I & Unit-II · Mid-Sem · BPIT / GGSIPU · B.Tech IT Sem VII

Part 1 – Questions (practise first)
Part 2 – Complete model answers
A. 1-mark · B. 2-mark · C. 3-mark · D. 5-mark
E. Long answers · F. Numericals · G. Diagrams
H. Comparisons · I. Case studies

Practice questions written from the syllabus and assignment emphasis — not leaked or guaranteed questions.

MID-SEM QUESTION BANK (Unit-I + Unit-II ONLY)

How to use: Part 1 has **only questions** – attempt them on paper first. Part 2 has **complete model answers** with the same numbers: every question from A to I is fully answered, with diagrams and tables written inside the answer. Tags: 📝 = question is based on a topic emphasized in your BPIT assignment (not copied) · 📘 = syllabus topic · ➕ = additional/exam-oriented. *These are practice questions written from the syllabus; not leaked or guaranteed questions.*

PART 1 — QUESTIONS

A. 1-MARK / VERY SHORT (30)

A1. Define a distributed system. A2. Name the three core characteristics of a distributed system. A3. What is transparency? (name any two types) A4. Give one example each of client–server and peer-to-peer systems. A5. What is grid computing? A6. Cloud computing follows which billing model? A7. What is middleware? A8. Name the two categories of system models. A9. What is latency? A10. Expand TCP and UDP. Which is connectionless? A11. What is an internetwork? A12. Name two IPC mechanisms. A13. What is marshalling? A14. What is a remote interface? A15. Which Java interface must every remote interface extend? A16. What is the default port of the RMI registry? A17. What is the role of a stub? A18. What does RPC stand for? A19. What is a notification? A20. What is an omission failure? A21. What is a Byzantine failure? A22. Define clock skew. A23. Define clock drift. A24. State the happened-before rules (names only). A25. State the Lamport rule for receiving a message. A26. What is a critical section? A27. Name two election algorithms. A28. What does the Bully algorithm elect? A29. What are the three conditions of consensus? A30. What is a consistent cut?

B. 2-MARK QUESTIONS (20)

B1. 📝 Define a distributed system and list four characteristics. B2. 📝 Differentiate between client–server and peer-to-peer in two points. B3. State any four advantages of distributed systems. B4. Why do we need system models? B5. 📝 Differentiate Architectural and Fundamental models (two points). B6. Write short note on synchronous and asynchronous distributed systems. B7. 📝 Why is internetworking needed? Name two devices used. B8. Distinguish TCP and UDP (two points). B9. What is distributed shared memory? B10. 📝 What are remote objects and remote object references? B11. What is the role of the RMI registry? B12. State the three invocation semantics. B13. What are events and notifications? Give an example. B14. 📝 What is meant by "no global clock"? Why is it a problem? B15. 📝 Explain process state and event. B16. Differentiate between Lamport and vector clocks (two points). B17. 📝 What are clock drift and skew? How are they related? B18. State the requirements of distributed mutual exclusion. B19. Give message complexity of Ricart–Agrawala, central server and Lamport ME algorithms. B20. What are the types of ordering in multicast?

C. 3-MARK QUESTIONS (12)

C1. 📝 Explain the characteristics of distributed systems with examples. C2. 📝 Explain client–server architecture with a diagram, advantages and disadvantages. C3. Explain the interaction model with latency, bandwidth and jitter. C4. Explain omission, arbitrary and timing failures. C5. 📝 Explain message passing as an IPC mechanism. C6. 📝 What is a stub and a skeleton? How does a remote call proceed? C7. Explain RPC with a neat diagram. C8. 📝 Explain the happened-before relation with an example. C9. 📝 Explain Cristian's algorithm with example. C10. 📝 What is global state? List its components. C11. Explain publish–subscribe model. C12. Explain reliable multicast properties.

D. 5-MARK QUESTIONS (12)

D1. 📝 Compare client–server, peer-to-peer, grid and cloud computing. D2. 📝 Explain architectural models of distributed systems with diagrams. D3. 📝 Explain networking and internetworking and the role of protocols in distributed applications. D4. 📝 Compare message passing and shared memory. D5. 📝 Explain distributed objects and RMI: remote object, remote interface, stub/proxy and invocation steps. D6. Compare RPC and RMI. D7. 📝 Explain Lamport logical clock with rules and example. D8. 📝 Explain need of physical clock synchronisation; discuss drift, skew, delay. D9. 📝 Explain Chandy–Lamport snapshot algorithm. D10. 📝 Explain Bully algorithm with example. D11. 📝 Explain Ricart–Agrawala algorithm. D12. 📝 Explain consensus and its conditions with an example.

E. 7 / 8 / 10-MARK LONG ANSWERS (8)

E1. 📝 Explain system models: need, architectural models and fundamental models with diagrams. E2. 📝 Design a simple Java RMI application (calculator): architecture, remote interface, server, client, registry, execution steps, code and output. E3. 📝 Explain Interprocess communication in detail; compare message passing and shared memory. E4. 📝 Explain global states, consistent cut and the distributed snapshot algorithm with a P1–P2–P3 example. E5. 📝 Explain distributed mutual exclusion: requirements and four algorithms with message complexity. E6. 📝 Explain election algorithms: Bully and Ring, with examples and comparison. E7. 📝 Explain consensus in distributed systems: requirements, process failures, communication delays, Byzantine generals, related problems. E8. Explain Synchronization: physical clock algorithms (Cristian, Berkeley, NTP) and logical clocks.

F. NUMERICAL / PROBLEM-BASED (8)

F1. 📝 P1: events a, send m1→P2 (b), receive m3 (c). P2: d, receive m1 (e), send m2→P3 (f). P3: g, receive m2 (h), send m3→P1 (i). Assign Lamport timestamps. F2. A process has clock 14 and receives a message timestamped 20; another has clock 30 and receives timestamp 25. Find new clocks. F3. Using Cristian's algorithm: T0 = 12:00:00.000, T1 = 12:00:00.600, server time = 12:00:10.200. Find client's new time and the accuracy bound. F4. 📝 Berkeley: master 2:00, slaves 1:50, 2:05, 2:20 (slave times as clock readings; master included in average). Find each adjustment. F5. Clock drift rate $\rho = 2\times10^{-6}$. Skew allowed 2 ms. How often to resynchronise? How much does a clock drift in one day? F6. Ricart–Agrawala with N = 8: messages per CS entry? Lamport ME? Central server? F7. Bully algorithm: processes P1–P6, P6 fails, P3 detects. List messages sent and the winner. Count messages. F8. Ring election: ring

P1→P4→P2→P6→P3→P1 (order as arranged), IDs = numbers, coordinator failed, P2 starts. Show messages until winner.

G. DIAGRAM-BASED (8)

G1. Draw and explain the RMI architecture (proxy, dispatcher, skeleton, communication module). G2. 📝 Draw the software-layer/middleware architecture of a distributed system. G3. Draw the message sequence for an RPC call. G4. 📝 Draw a timeline showing happened-before and concurrent events for 3 processes. G5. 📝 Draw a consistent and an inconsistent cut. G6. 📝 Draw message flow of Chandy–Lamport snapshot for P1, P2, P3. G7. 📝 Draw message flow of Bully election with 5 processes. G8. Draw the hold-back queue for ordered multicast.

H. COMPARE / DIFFERENTIATE (10)

H1. 📝 Client–server vs peer-to-peer. H2. 📝 Message passing vs shared memory. H3. 📝 RMI vs RPC. H4. Synchronous vs asynchronous DS. H5. 📝 Logical vs physical time. H6. Lamport vs vector clocks. H7. 📝 Bully vs Ring algorithm. H8. Cristian vs Berkeley. H9. Token-based vs non-token-based mutual exclusion. H10. Crash vs Byzantine failures.

I. CASE-STUDY QUESTIONS (5)

I1. 📝 A college wants a Student Information Service that lets any lab computer call a method to fetch a student's record from a central server. Design it using Java RMI. I2. 📝 Three bank branch servers P1, P2, P3 hold balances 500, 300, 200. Design a method to record the total consistently while transfers are happening. I3. 📝 Five replicated database servers; one may crash. How do they agree to commit or abort? Discuss challenges. I4. A file server is shared by 6 clients that all update the same file. Choose and justify a mutual-exclusion scheme. I5. In a cluster of 7 servers the coordinator crashes. Show how a new coordinator is chosen with the Bully algorithm and with the ring algorithm.

PART 2 — MODEL ANSWERS

A. 1-MARK ANSWERS

A1. A system whose components on networked computers communicate and coordinate **only by passing messages**. A2. Concurrency, no global clock, independent failures. A3. Hiding the distribution of components from users; e.g., access transparency, location transparency (also replication, failure, mobility, concurrency, performance, scaling). A4. Client–server: web browser–web server. P2P: BitTorrent. A5. Pooling geographically distributed heterogeneous resources of multiple organisations into a virtual supercomputer. A6. Pay-per-use (on-demand, metered). A7. Software layer between OS and applications masking heterogeneity (e.g., RMI, CORBA). A8. Architectural models and fundamental models. A9. Delay between start of transmission at sender and start of receipt at receiver. A10. Transmission Control Protocol, User Datagram Protocol; UDP is connectionless. A11. A network of networks connected by routers acting as a single logical network. A12. Message passing and shared memory (sockets, message queues, DSM). A13. Converting data structures into a flat byte sequence for transmission (and reverse = unmarshalling). A14. Specification of the methods of a remote object that clients may invoke remotely. A15. `java.rmi.Remote`. A16. 1099. A17. Client-side proxy that marshals the call and forwards it to the server, hiding remoteness. A18. Remote Procedure Call. A19. A message describing an event, sent to interested subscribers. A20. A process or channel fails to perform an action (crash or message loss). A21. Arbitrary behaviour: wrong values, contradictory messages, corruption. A22. Difference between readings of two clocks at the same instant. A23. Rate at which a clock deviates from a perfect reference clock. A24. (1) Same-process order, (2) send → receive, (3) transitivity. A25. `L = max(L, t) + 1`. A26. Code segment that accesses a shared resource and must be executed by one process at a time. A27. Bully and Ring algorithms. A28. The alive process with the highest ID. A29. Termination, agreement, validity (integrity). A30. A cut in which every receive event's send event is also inside the cut (no orphan messages).

B. 2-MARK ANSWERS

B1. Def (A1). Characteristics: concurrency (many users act simultaneously), no global clock (each node has own clock), independent failures (one node can fail alone), resource sharing/heterogeneity/openness/scalability. B2. C-S: fixed roles, central server (SPOF). P2P: symmetric roles, decentralised, no SPOF. B3. Resource sharing, scalability, reliability/availability, better performance, cost-effectiveness, geographical distribution. B4. To make assumptions explicit, identify common design issues, reason about correctness/feasibility of algorithms, and generalise across systems. B5. Architectural: structure/placement of components (client-server, P2P). Fundamental: abstract properties – interaction, failure, security. B6. Synchronous: known bounds on execution time, message delay, drift → timeouts reliable. Asynchronous: no bounds (Internet) → can't distinguish slow from crashed. B7. To connect heterogeneous networks so hosts communicate globally (Internet). Devices: router, gateway (also bridge/switch). B8. TCP – connection-oriented, reliable, ordered. UDP – connectionless, unreliable, faster. B9. DSM gives the illusion of one shared address space over physically separate memories using page faults and message passing. B10. Remote object: object whose methods can be invoked from other processes. Remote object reference: system-wide identifier (address, port, time, object number, interface) used to invoke it. B11. Name service: server binds a name to the stub, client looks it up to obtain a stub (port 1099). B12. Maybe, at-least-once, at-most-once (Java RMI: at-most-once). B13. Event: occurrence of interest (e.g., stock price change). Notification: message about it sent to subscribers via event service. B14. Each computer has its own clock; clocks drift, so events on different machines can't be ordered by their timestamps → need logical clocks/sync algorithms. B15. State = values of a process's variables; event = action (internal/send/receive) that changes state. B16. Lamport: one integer, $a \to b \Rightarrow L(a) < L(b)$ only. Vector: N integers, $a \to b \Leftrightarrow V(a) < V(b)$, detects concurrency. B17. Skew = difference of two clocks at an instant; drift = rate of deviation. Drift makes skew grow with time (skew $\leq 2\rho\Delta t$). B18. Safety (one in CS), liveness (no deadlock/starvation), ordering/fairness (happened-before order). B19. RA = $2(N-1)$; central = 3; Lamport = $3(N-1)$. B20. FIFO, causal, total ordering.

C. 3-MARK ANSWERS (complete)

C1. Explain the characteristics of distributed systems with examples.

Definition: A distributed system is a system in which components located on networked computers communicate and coordinate their actions **only by passing messages**.

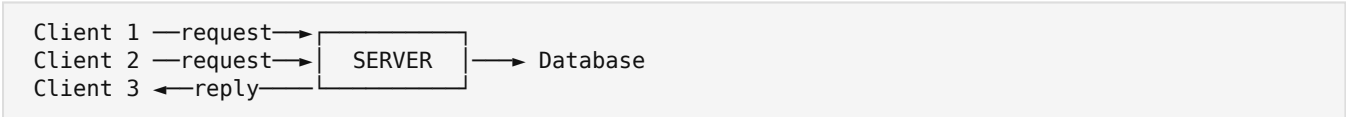
Characteristics: 1. **Concurrency:** Many programs run at the same time on different computers and share resources. *Example:* thousands of users booking IRCTC tickets at the same moment. 2. **No global clock:** Every computer has its own clock and clocks drift apart, so there is no single notion of "correct time". Coordination is done by messages. *Example:* two bank servers stamp the same transfer with different times. 3. **Independent failures:** One computer or network link can fail while others keep running, and the others may not even know about the failure. *Example:* one Google server crashes but search still works. 4. **Resource sharing:** Hardware, software and data are shared over the network. *Example:* shared printer, Google Drive. 5. **Heterogeneity:** Different hardware, OS, networks and languages work together (hidden by middleware). *Example:* Android phone talking to a Linux server. 6. **Openness:** New services can be added using published interfaces. *Example:* HTTP/REST APIs. 7. **Scalability:** The system keeps working well as users grow. *Example:* YouTube. 8. **Transparency:** The user sees one system, not many computers. *Example:* a URL doesn't reveal the server's location.

Conclusion: The first three (concurrency, no global clock, independent failures) are the core characteristics and are the reason synchronization algorithms (Unit-II) are needed.

C2. Explain client–server architecture with a diagram, advantages and disadvantages.

Definition: Client–server is an architecture in which **client processes send requests** to **server processes**, which manage resources and **send back replies**.

Diagram:



Working: 1. The client sends a request (for example an HTTP GET for a web page). 2. The server receives the request and processes it (possibly querying a database). 3. The server sends a reply with the result. 4. The client displays or uses the result.

Advantages: - Central control of data, so security and backup are easy. - Clear roles make the system easy to design and manage. - Clients can be simple (thin).

Disadvantages: - The server is a **single point of failure**. - The server becomes a **bottleneck** under heavy load. - Scaling needs expensive powerful servers or replication.

Examples: web browser and web server, e-mail (SMTP/IMAP), ATM and bank server, database server.

C3. Explain the interaction model with latency, bandwidth and jitter.

Definition: The interaction model describes **how processes interact by messages**, the **performance of communication channels**, and the problem of **time** in a distributed system.

1. Performance of communication channels - **Latency:** delay between the start of a message's transmission by the sender and the start of its receipt by the receiver. It includes network delay, operating-system delay and queueing. - **Bandwidth:** total amount of data that can be transmitted per unit time (for example 100 Mbps). - **Jitter:** variation in the time taken to deliver a series of messages. It matters for audio and video, where frames must arrive at regular intervals. - Transmission time ≈ **latency + (message length ÷ bandwidth)**.

2. Computer clocks: each process has its own clock with a **drift rate**, so local times differ and there is no global time.

3. Two variants: | Synchronous system | Asynchronous system | |---|---| | Known bounds on execution time, message delay and clock drift | No bounds on any of these | | Timeouts can detect failure | Cannot tell a slow process from a crashed one | | Example: real-time control | Example: the Internet |

4. Event ordering: because clocks are not reliable, events are ordered by causality (happened-before, logical clocks).

C4. Explain omission, arbitrary and timing failures.

The **failure model** defines the ways in which processes and channels can fail.

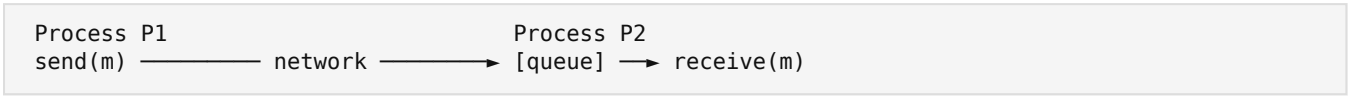
Failure	Description	Example
Omission – process (crash)	Process halts and does nothing more. If others can detect it reliably it is called *fail-stop*	Server power failure
Omission – channel	Message is lost: *send-omission* (lost in sender buffer), *channel omission* (lost in network), *receive-omission* (lost in receiver buffer)	Packet dropped by a congested router
Arbitrary (Byzantine)	Process or channel behaves in any way: sends wrong or contradictory values, corrupts or duplicates messages, omits steps. Worst type	Hacked node sending false data
Timing	Response is outside the allowed time interval: clock failure, performance failure. Only meaningful in **synchronous** systems	Video frame arrives too late

Masking failures: a system can hide failures, for example replication hides a crash, and a checksum converts a corrupted message into an omission (the message is discarded and re-sent). **Reliable communication** = **validity** (every sent message is eventually delivered) + **integrity** (the received message is identical to the one sent, with no duplicates).

C5. Explain message passing as an IPC mechanism.

Definition: In message passing, processes communicate by **explicitly sending and receiving messages** over a channel; they share no variables. The primitives are `send(destination, message)` and `receive(source, message)`.

Diagram:



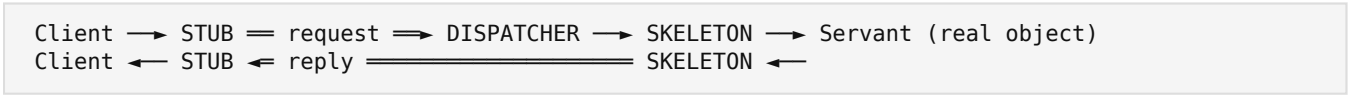
Important concepts: - **Synchronous (blocking):** the sender waits until the message is received; receive waits until a message arrives. - **Asynchronous (non-blocking):** the sender continues after the message is copied to a buffer. - **Sockets:** endpoint = (IP address, port). **UDP datagram sockets** are unreliable and have no connection; **TCP stream sockets** are reliable and ordered. - **Marshalling:** converting data into a byte stream in an agreed external format (Java serialization, XML, JSON) so different machines understand it. - **Direct or indirect:** directly to a process, or via a mailbox or message queue.

Advantages: works between different computers; processes are isolated from each other (more secure); synchronisation is built into send/receive; scalable. **Limitations:** slower because of copying and network delay; the programmer must handle message formats, lost messages and failures. **Applications:** chat applications, client–server systems, MPI for parallel programs, message queues (RabbitMQ, Kafka).

C6. What is a stub and a skeleton? How does a remote call proceed?

- **Stub (proxy):** client-side local object that implements the same remote interface as the remote object. It **marshals** the method identifier and arguments into a request message, sends it, waits, and **unmarshals** the reply. It makes the remote call look local.
- **Skeleton:** server-side object that receives the request, **unmarshals** the arguments, calls the method on the real object (servant), then **marshals** the result and sends it back. A **dispatcher** chooses the right skeleton method.

Diagram:

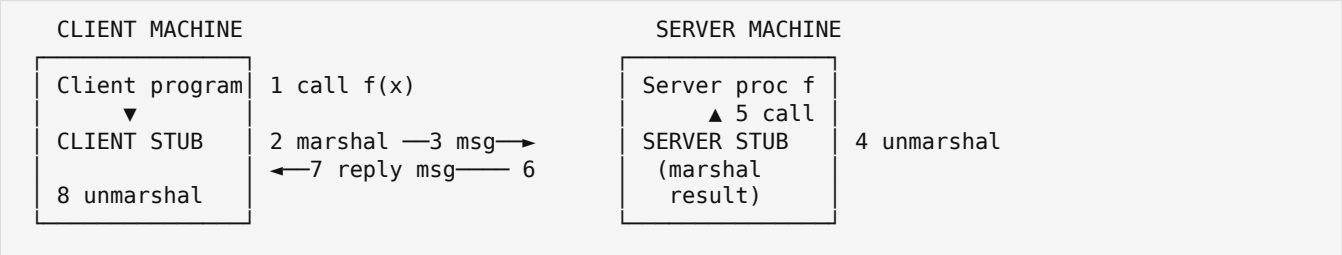


Steps of a remote call: 1. The client calls a method on the stub like a normal call. 2. The stub marshals the method id and parameters. 3. The communication module sends the request over the network. 4. At the server, the dispatcher selects the method and passes the request to the skeleton. 5. The skeleton unmarshals the arguments and invokes the servant. 6. The servant executes and returns the result to the skeleton. 7. The skeleton marshals the result and sends the reply. 8. The stub unmarshals the result and returns it to the client.

C7. Explain RPC with a neat diagram.

Definition: Remote Procedure Call (RPC) allows a program to **call a procedure that executes on another computer** as if it were a local procedure; the network communication is hidden by stubs.

Diagram:



Steps: 1. The client calls the client stub with normal parameters. 2. The client stub marshals the parameters into a message. 3. The RPC runtime and OS send the message to the server. 4. The server stub unmarshals the parameters. 5. The server stub calls the actual procedure. 6. The procedure returns its result, and the server stub marshals it. 7. The reply message is sent back. 8. The client stub unmarshals the result and returns it to the client.

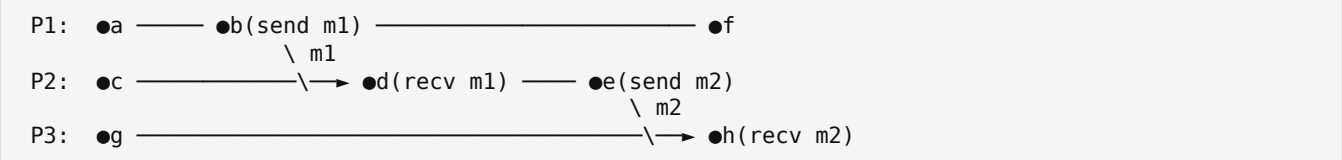
Supporting parts: IDL (Interface Definition Language) describes procedures, and an IDL compiler (for example rpcgen) generates the stubs. A **binder/portmapper** helps the client locate the server. **XDR** is a common data representation. **Issues:** no pointers (pass by value), failures (lost request or reply, server crash), invocation semantics (maybe, at-least-once, at-most-once). **Examples:** Sun RPC (used by NFS), gRPC.

C8. Explain the happened-before relation with an example.

Definition (Lamport, 1978): The happened-before relation (→) captures the **causal order** of events in a distributed system. It is defined by three rules: 1. **HB1:** if a and b are events in the **same process** and a occurs before b, then a → b. 2. **HB2:** if a is the **sending** of message m and b is the **receipt** of m, then a → b. 3. **HB3 (transitivity):** if a → b and b → c, then a → c.

If neither a → b nor b → a, the events are **concurrent** (a ∥ b).

Example:



- a → b (HB1), b → d (HB2, message m1), d → e (HB1), e → h (HB2, m2).
- By transitivity: a → d, a → h, b → h.
- **Concurrent:** a ∥ c (no path between them), f ∥ d, g ∥ b.

Key point: → is a **partial order**; it shows potential causality, not real time.

C9. Explain Cristian's algorithm with example.

Purpose: external clock synchronisation of a client with a **time server** that receives UTC. **Assumption:** the round-trip time is short compared with the required accuracy, and the request and reply take roughly equal time.

Steps: 1. The client records its local time **T0** and sends a request to the time server. 2. The server replies with its current time **Ts**. 3. The client records the reply arrival time **T1**. Round trip time RTT = T1 − T0. 4. The client sets its clock to **Ts + RTT/2**. 5. Accuracy = ± (RTT/2 − min), where min is the minimum one-way delay.

Diagram:

```
Client                                Time server
 T0 ──── request ──────────────────►
    ◄──────── reply (Ts) ────────────
 T1        new time = Ts + (T1 − T0)/2
```

Example: T0 = 10:00:00.000, T1 = 10:00:00.800, Ts = 10:00:05.300. RTT = 0.8 s → new time = 10:00:05.300 + 0.400 = **10:00:05.700**; accuracy ± 0.4 s (taking min ≈ 0).

Limitations: the server is a single point of failure, and a faulty server gives wrong time. Fixes: use several servers, and take the reading with the smallest RTT.

C10. What is global state? List its components.

Definition: The global state of a distributed system is the **set of local states of all processes together with the states of all communication channels** at some instant.

Components: 1. **Local state of each process (S1, S2, …, Sn):** the values of its variables, for example the account balance at each bank server. 2. **State of each channel (Cij):** the messages that have been **sent but not yet received** (in transit).

GS = {S1, S2, …, Sn} ∪ {Cij for every channel}.

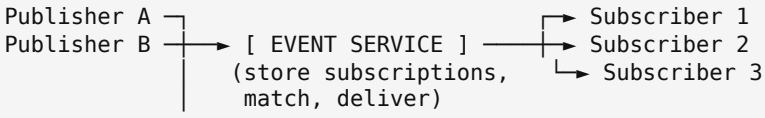
Consistent global state: a state that could actually have happened. It corresponds to a **consistent cut**: every message received in the state was also sent in the state (no orphan message).

Why it is needed: deadlock detection, termination detection, distributed garbage collection, checkpointing and recovery, and debugging. **Difficulty:** there is no global clock, so all processes cannot record their states at the same instant. The Chandy–Lamport snapshot algorithm solves this.

C11. Explain the publish–subscribe model.

Definition: Publish–subscribe is a **distributed event-based** model in which **publishers** generate events, **subscribers** register interest in types of events, and an **event service** delivers **notifications** of matching events to subscribers.

Diagram:



Working: 1. Subscribers `subscribe(event type or filter)` with the event service. 2. A publisher `publish(event)`. 3. The event service matches the event against subscriptions. 4. It `notify`s every matching subscriber, asynchronously.

Types of subscription: topic/channel-based, content-based (filter on attributes, for example price > 500), type-based. **Features:** asynchronous; publishers and subscribers do not know each other (decoupled in space and time); heterogeneous. **Advantages:** loose coupling, scalable, real-time updates, no polling. **Disadvantages:** delivery and ordering guarantees are hard; the broker can be a bottleneck; hard to debug. **Examples:** stock price alerts, YouTube and mobile push notifications, MQTT for IoT, Apache Kafka.

C12. Explain reliable multicast properties.

Multicast sends one message to all members of a group. **Reliable multicast** gives the following guarantees (Coulouris): 1. **Integrity:** a correct process delivers a message **at most once**, and only if it was actually multicast by some process (no duplicates, no spurious messages). 2. **Validity:** if a correct process multicasts message m, it will **eventually deliver m** itself. 3. **Agreement:** if a correct process delivers m, then **all other correct processes** in the group eventually deliver m ("all or none").

Implementation (R-multicast over B-multicast):

```
To R-multicast m:          B-multicast(g, m)
On B-deliver(m) at q:      if m not received before:
                               if q ≠ sender: B-multicast(g, m)   // re-send to all
                               R-deliver(m)
```

Because every receiver re-multicasts m, even if the original sender crashes midway, all correct processes still get m. **Cost:** $O(N^2)$ messages. More efficient versions use sequence numbers, piggy-backed acknowledgements and negative acknowledgements (NACKs).

D. 5-MARK ANSWERS (complete)

D1. Compare client–server, peer-to-peer, grid and cloud computing.

Introduction: These are four common examples (styles) of distributed systems. They differ in who controls resources, the roles of the nodes, and their purpose.

Brief definitions: - **Client–server:** clients request services from central servers. *Example:* web browser and web server. - **Peer-to-peer (P2P):** every node (peer) is both client and server and shares resources directly. *Example:* BitTorrent. - **Grid computing:** geographically spread, heterogeneous computers of many organisations are pooled to work like a virtual supercomputer on large problems. *Example:* CERN LHC Computing Grid, SETI@home. - **Cloud computing:** computing services (servers, storage, software) are delivered over the Internet on demand, on a pay-per-use basis. *Example:* AWS, Microsoft Azure, Gmail.

Diagrams:



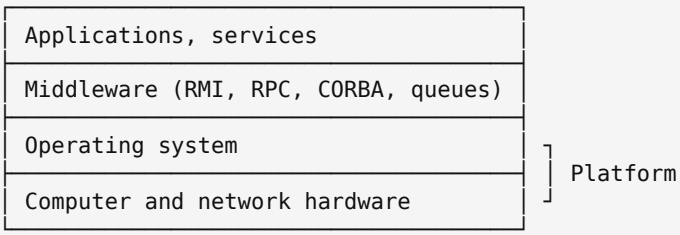
Comparison table: | Feature | Client–Server | Peer-to-Peer | Grid | Cloud | |---|---|---|---|---| | Control | Centralised (server) | Decentralised | Distributed across organisations (virtual organisation) | Centralised by provider | | Roles | Fixed: client and server | Equal: every peer is both | Resource providers and users | Provider and customer | | Main purpose | Access services and data | Share files and resources | Solve very large computations | Rent IT resources on demand | | Scalability | Limited by server | High | Very high | Very high, elastic | | Single point of failure | Yes (server) | No | Scheduler, partly | No (redundant data centres) | | Ownership | One organisation | Individual users | Many organisations | Cloud provider | | Resources | Mostly homogeneous | Heterogeneous | Heterogeneous | Virtualised pool | | Cost model | Own servers | Free | Shared/project based | Pay-per-use | | Security | Easy to control | Hard | Hard (trust between organisations) | Managed by provider | | Applications | Web, e-mail, banking | BitTorrent, blockchain | Scientific simulation, genome research | Web hosting, SaaS apps, storage |

Conclusion: Client–server is simple but centralised; P2P removes the central server; grid pools the power of many organisations for big jobs; cloud offers elastic, pay-per-use services and is the most common today.

D2. Explain architectural models of distributed systems with diagrams.

Definition: An architectural model describes the **structure** of a distributed system: its **components**, where they are **placed** on the network, and their **relationships and roles**.

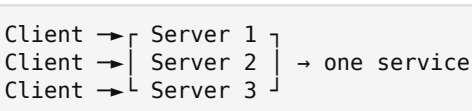
1. Software layers and middleware



Middleware hides heterogeneity and gives one programming model to applications.

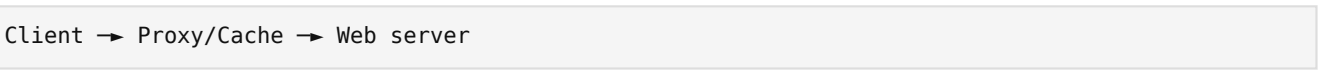
2. Client–server: clients send requests, servers reply. A server can itself be a client of another server (for example a web server calling a database).

3. Multiple servers: one service is provided by several servers that cooperate by **partitioning** data (each holds part of it) or **replicating** it (each holds a copy, for availability).



Example: DNS, Google search clusters.

4. Proxy servers and caches: a cache keeps recently used objects close to the client; a proxy server shares a cache among many clients. This reduces latency and load.



5. Peer processes (P2P): all processes have similar roles and cooperate without a server (BitTorrent).

6. Mobile code: code is downloaded from a server and runs at the client (JavaScript, applets). It gives fast interaction but carries security risk.

7. Mobile agents: a running program (code + data) moves from computer to computer to do a task for the user. It reduces communication but is hard to secure.

8. Thin clients: the client only displays; the processing happens on a powerful server (cloud desktop).

Conclusion: The choice of architecture affects performance, reliability, scalability and security. Most real systems combine several of these models.

D3. Explain networking and internetworking and the role of protocols in distributed applications.

Definitions: - **Network:** a set of computers and devices connected so that they can exchange data. - **Internetwork:** a **network of networks**: many different networks joined by **routers/gateways** so they work as one logical network (the Internet). - **Protocol:** a set of rules for the format, order and meaning of messages exchanged between computers.

Types of network: | Type | Range | Example | |---|---|---| | LAN | Building or campus | Ethernet, Wi-Fi | | MAN | City | Metro Ethernet | | WAN | Country or world | Internet backbone | | Wireless |

Varied | Wi-Fi, 4G/5G |

Protocol layering (TCP/IP):

```
Application   HTTP, FTP, SMTP, DNS        ← messages
Transport     TCP (reliable) / UDP (fast) ← ports, end-to-end
Internet      IP (addressing, routing)    ← packets
Link          Ethernet, Wi-Fi             ← frames
```

Each layer uses the services of the layer below, which hides complexity.

Internetwork diagram:

```
LAN-1 [PC]—[PC]—[Router]═ WAN ═[Router]—[Laptop]—[Phone] LAN-2
```

Role in distributed applications: 1. **Connectivity:** carries the messages between components; without a network there is no distributed system. 2. **Protocols give a common language:** heterogeneous machines can talk (IP addresses, standard formats). 3. **TCP gives reliability** (ordering, retransmission) for RPC/RMI and the web; **UDP gives speed** for streaming and DNS. 4. **Routing** lets messages cross many networks worldwide. 5. **Performance parameters** such as latency, bandwidth and jitter directly affect distributed applications. 6. **Security protocols** (TLS/SSL) protect messages.

Conclusion: Networks and internetworks provide the communication infrastructure, and protocols make communication standard, reliable and possible between different systems.

D4. Compare message passing and shared memory.

Message passing: processes communicate by explicit `send` and `receive` of messages; no memory is shared. **Shared memory:** processes communicate by reading and writing a **common memory area**, with locks or semaphores for synchronisation. In a distributed system it is emulated by **Distributed Shared Memory (DSM)**.

Diagrams:

```
Message passing:   P1 ── send(m) ──network──► receive(m) P2
Shared memory:     P1 ──write──► [ SHARED MEMORY ] ◄──read── P2
```

Comparison: | Point | Message passing | Shared memory | |---|---|---| | Working | Explicit send/receive | Read/write common variables | | Location of processes | Same or different computers | Same computer (or DSM) | | Speed | Slower (copying, network) | Faster (direct access) | | Synchronisation | Built in (blocking receive) | Programmer uses locks/semaphores | | Data copy | Data copied between address spaces | No copy needed | | Main risk | Lost messages, format errors | Race conditions, data corruption | | Scalability | High | Low | | Isolation/security | Good | Poor | | Implementation | Sockets, MPI, message queues, RPC/RMI | shm, mmap, threads, DSM | | Applications | Client–server apps, chat, clusters | Multithreaded programs, producer–consumer buffer |

Advantages and limitations: - Message passing works over networks and is safer, but is slower and needs the programmer to handle messages. - Shared memory is fast and simple to use, but only works on one machine and needs careful locking.

Conclusion: Distributed systems mainly use **message passing** because separate computers have no physical common memory.

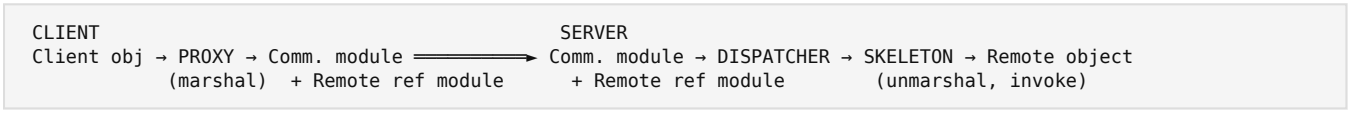
D5. Explain distributed objects and RMI: remote object, remote interface, stub/proxy and invocation steps.

Distributed object: an object whose methods can be called by processes on **other computers**. The object model is extended across the network.

Key terms: - **Remote object:** an object that can receive remote invocations. - **Remote object reference:** a unique identifier for the remote object across the whole system (IP address + port + time + object number + interface). It can be passed as an argument or result. - **Remote interface:** lists the methods of a remote object that can be called remotely (in Java, an interface extending `Remote`). - **Stub/proxy:** the client-side representative that marshals calls. - **Skeleton/dispatcher:** the server side, which unmarshals and calls the object.

RMI (Remote Method Invocation): calling a method of a remote object as if it were local.

Architecture:



Invocation steps: 1. The client calls the method on the proxy. 2. The proxy marshals the method id and arguments into a request. 3. The communication module sends the request. 4. The server's remote reference module finds the local object. 5. The dispatcher selects the method and the skeleton unmarshals the arguments. 6. The skeleton calls the remote object's method. 7. The result is marshalled into a reply and sent back. 8. The proxy unmarshals the result and gives it to the client.

Differences from a local call: parameters are copied (no pointers); there are extra failures (`RemoteException`); the semantics are at-most-once in Java RMI. **Example:** `double r = calc.add(5, 3);` executes on the server and returns 8.

D6. Compare RPC and RMI.

RPC: calls a **procedure** on a remote machine as if it were local (procedural style). **RMI:** calls a **method of a remote object** (object-oriented style). RMI is the OO counterpart of RPC.

Similarities: both hide the network with stubs, use marshalling and the request–reply protocol, and have the same failure semantics issues.

Feature	RPC	RMI
Programming paradigm	Procedural	Object-oriented
What is called	Remote function/procedure	Method of a remote object

Feature	RPC	RMI
Parameters	Simple data types, by value; no pointers	Objects (serialised, by value) and remote object references
Language	Language-independent (C, via IDL)	Java RMI works only Java-to-Java
Interface definition	IDL file (rpcgen)	Java interface extending `Remote`
Client/server components	Client stub / server stub	Stub (proxy) / skeleton
Naming service	Portmapper / binder	RMI registry (port 1099)
Object features	None	Inheritance, polymorphism, dynamic class loading
Garbage collection	Not needed	Distributed garbage collection
Overhead	Lower	Higher (serialisation)
Security	Depends on implementation	Java security manager
Example	Sun RPC used by NFS, gRPC	Java RMI, CORBA (similar idea)

Conclusion: RPC suits simple procedural services in any language; RMI suits Java object-oriented applications where objects and references need to be passed.

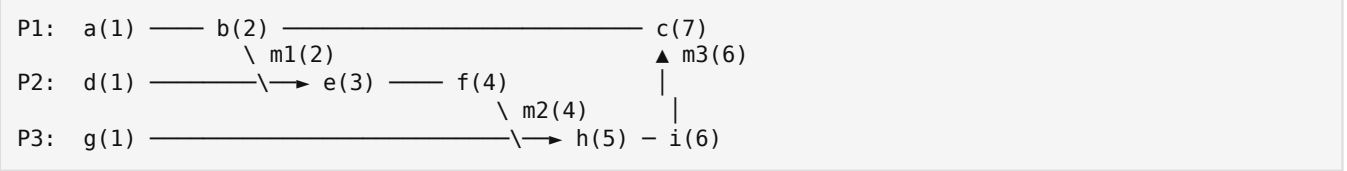
D7. Explain Lamport logical clock with rules and example.

Definition: A Lamport logical clock is a counter L at each process that gives every event a timestamp such that **if a → b then L(a) < L(b)**. It orders events by causality without physical clocks.

Why needed: distributed systems have no global clock, and physical clocks drift, so real time cannot order events on different machines.

Rules: - **LC1:** before every event (internal or send), L = L + 1. - **LC2:** when sending message m, attach the timestamp t = L. - **LC3:** when receiving (m, t): L = max(L, t) + 1.

Example (clocks start at 0):



Event	Calculation	L
a	0 + 1	1
b (send m1)	1 + 1	2
d	0 + 1	1
e (recv m1, t = 2)	max(1, 2) + 1	3
f (send m2)	3 + 1	4

Event	Calculation	L
g	0 + 1	1
h (recv m2, t = 4)	max(1, 4) + 1	5
i (send m3)	5 + 1	6
c (recv m3, t = 6)	max(2, 6) + 1	7

Properties: - a → b ⇒ L(a) < L(b). - The converse is **false**: L(d) = 1 < L(b) = 2 but d and b are concurrent. - **Total order:** break ties using the process id, (L, pid).

Limitations: cannot detect concurrent events (vector clocks solve this); timestamps are not real time.

D8. Explain the need for physical clock synchronisation; discuss drift, skew and delay.

Need: 1. Correct **timestamps** of transactions (banking, e-commerce, stock trading). 2. Ordering of **log files** when auditing or debugging. 3. **Security:** Kerberos tickets and certificates have expiry times. 4. **File systems and tools:** `make` compares file modification times. 5. **Real-time scheduling** and timeouts.

Clock skew: the difference between the readings of two clocks at the same instant. *Example:* C1 = 10:00:05, C2 = 10:00:02 → skew = 3 s.

Clock drift: the rate at which a clock gains or loses time compared with a perfect clock, caused by the quartz crystal, temperature and ageing. A typical drift rate is $\rho \approx 10^{-6}$ s/s, which is about **86.4 ms per day**. - Drift causes skew to grow: two clocks can differ by up to **2ρΔt** after time Δt. - To keep skew below δ, resynchronise at least every **δ / 2ρ** seconds. *Example:* δ = 1 ms, $\rho = 10^{-6}$ → every 500 s.

Communication delay: the time value sent by a server is already old when it arrives, and the delay is variable and unknown (queueing, routing, asymmetric paths). So the receiver cannot know the exact current server time.

Challenges: - Clocks cannot be set backwards (this would break ordering), so they are slowed down gradually instead. - Time servers may fail or give wrong time. - Network delays are unpredictable. - Perfect synchronisation is impossible.

Solutions: - **Cristian's algorithm:** time = Ts + RTT/2. - **Berkeley algorithm:** the master averages the clocks of the group. - **NTP:** a hierarchy of time servers across the Internet. - **Logical clocks:** when only the order of events matters.

D9. Explain the Chandy–Lamport snapshot algorithm.

Purpose: to record a **consistent global state** (process states + channel states) of a running distributed system without stopping it.

Assumptions: channels are FIFO, reliable (no loss) and one-directional; the graph of processes is strongly connected; there are no failures during the snapshot; any process can start the

snapshot. A special control message called a **MARKER** is used.

Algorithm: 1. **Initiator:** records its own local state, sends a MARKER on every outgoing channel, and starts recording messages on every incoming channel. 2. **On receiving a MARKER on channel c:** - **If the process has not yet recorded its state:** it records its state, sets the state of c = empty, sends MARKERs on all outgoing channels, and starts recording its other incoming channels. - **Otherwise:** it stops recording c. The state of c = all messages received on c since it recorded its own state. 3. The snapshot ends when every process has received a MARKER on every incoming channel.

Example (bank): P1 = ₹500, P2 = ₹300, P3 = ₹200 (total ₹1000). 1. P1 sends ₹50 to P2 (m1), so P1 = 450. P1 then starts the snapshot: records **S1 = 450** and sends markers. 2. P3 sends ₹20 to P1 (m2), so P3 = 180, before P3 gets a marker. 3. P2 receives m1 (P2 = 350), then P1's marker: records **S2 = 350**, C12 = ∅. 4. P3 receives the marker: records **S3 = 180**. 5. P1 receives m2 while recording C31, then P3's marker: **C31 = {₹20}**.

```
P1 — send m1 —[record 450, MARKER→]——— recv m2 — recv MARKER(C31)
P2 ————————— recv m1 — recv MARKER —[record 350, MARKER→]
P3 — send m2 — recv MARKER —[record 180, MARKER→]
```

Recorded: 450 + 350 + 180 + 20 (in transit) = **₹1000** ✓ consistent.

Complexity: one marker per channel, so O(E) messages (N(N−1) if fully connected). **Uses:** deadlock detection, termination detection, checkpointing. **Limitation:** needs FIFO channels; the recorded state may not have actually occurred at any single instant, but it is a consistent, reachable state.

D10. Explain Bully algorithm with example.

Purpose: to elect a new **coordinator** (the alive process with the **highest ID**) when the current one fails.

Assumptions: the system is synchronous (timeouts are reliable); every process knows the IDs of all other processes; message delivery is reliable; processes fail only by crashing.

Messages: ELECTION (starting an election), **OK/ANSWER** (reply from a higher process), **COORDINATOR** (announcing the winner).

Steps: 1. Process P notices the coordinator has failed and sends ELECTION to all processes with a **higher ID**. 2. If no one answers within the timeout, P wins and sends COORDINATOR to all processes. 3. If a higher process answers OK, P stops and waits for a COORDINATOR message. 4. Every higher process that received ELECTION replies OK and starts its own election. 5. Finally the highest alive process gets no answer and announces itself as coordinator.

Example (P1–P5, P5 crashes, P2 notices):

```
P2 —ELECTION→ P3, P4, P5(✗)
P3 —OK→ P2      P4 —OK→ P2               (P2 stops)
P3 —ELECTION→ P4, P5(✗)      P4 —OK→ P3  (P3 stops)
P4 —ELECTION→ P5(✗) ..... timeout, no reply
P4 —COORDINATOR(4)→ P1, P2, P3           (P4 is the new coordinator)
```

Message complexity: - **Best case:** the second-highest process detects the failure and sends **N − 2** COORDINATOR messages. - **Worst case:** the lowest process detects the failure, giving **O(N²)** messages.

Advantages: simple; the highest-ID process always wins; handles recovered processes (a recovered high-ID process "bullies" its way back). **Disadvantages:** many messages; needs a synchronous system; every process must know all IDs; a wrong timeout can produce two coordinators.

D11. Explain the Ricart–Agrawala algorithm.

Purpose: distributed mutual exclusion **without a token or coordinator**, using multicast and **Lamport timestamps**.

Assumptions: N processes with unique IDs; each process knows all the others; reliable message delivery; no failures. **States of a process:** RELEASED (not interested), WANTED (wants the critical section, CS), HELD (inside the CS).

Algorithm:

```
Init:      state = RELEASED
Enter CS:  state = WANTED
           multicast REQUEST(T, pi) to all other processes
           wait until N−1 REPLY messages are received
           state = HELD
On REQUEST(Tj, pj) at pi:
           if state == HELD or (state == WANTED and (Ti, pi) < (Tj, pj))
                 queue the request (defer the reply)
           else  send REPLY to pj
Exit CS:   state = RELEASED
           send REPLY to all queued requests
```

The smaller timestamp has higher priority; ties are broken by the smaller process id.

Example (P1, P2, P3): P1 requests with (5, P1) and P2 requests with (7, P2) at the same time; P3 is RELEASED. 1. P3 replies to both. 2. P2 receives P1's request: (7, P2) > (5, P1), so P2 replies to P1. 3. P1 receives P2's request: (5, P1) < (7, P2), so P1 **defers** its reply. 4. P1 now has 2 replies and **enters the CS**. 5. When P1 exits, it sends the deferred REPLY to P2, and **P2 enters the CS**.

```
P1 —REQ(5)—► P2, P3        P2 —REQ(7)—► P1, P3
P3 —REPLY—► P1, P2         P2 —REPLY—► P1   ⇒ P1 enters CS
P1 exits —REPLY—► P2                        ⇒ P2 enters CS
```

Message complexity: 2(N − 1) per entry (N − 1 requests + N − 1 replies). Synchronisation delay = 1 message time. **Advantages:** fully distributed; fair (timestamp order); free from deadlock and starvation; no separate release messages. **Disadvantages:** many messages; the crash of any process blocks others (no reply); every process takes part in every decision.

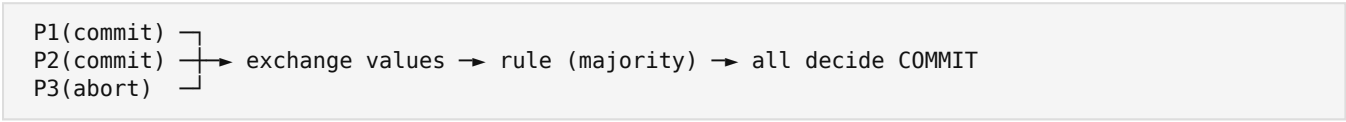
D12. Explain consensus and its conditions with an example.

Definition: In the consensus problem, each process **proposes a value**, and all correct processes must **agree on (decide) the same single value**, even if some processes fail.

Need: committing or aborting a distributed transaction, keeping replicas consistent, leader election, total-order multicast, blockchain, fault-tolerant control (aircraft computers).

Conditions: 1. **Termination:** every correct process eventually decides a value. 2. **Agreement:** all correct processes decide the **same** value. 3. **Validity (integrity):** if all correct processes proposed the same value v, then the decision is v. (The decided value must be one that was proposed.)

Example: three database replicas decide whether to commit a transaction. P1 = commit, P2 = commit, P3 = abort. They exchange votes and apply the same rule (majority), so all decide **commit**. - Agreement ✓: all decide commit. - Validity ✓: commit was proposed. - Termination ✓: all decide after the exchange.



Solution with crash failures (synchronous system): with f possible crashes, run **f + 1 rounds**. In each round every process multicasts the new values it has learned; at the end each decides with the same function (for example the minimum or majority).

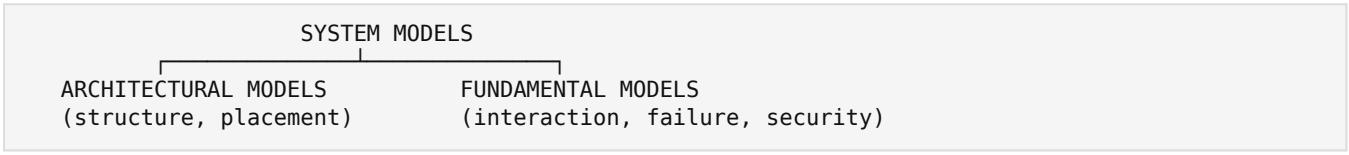
Difficulties: crashes cause partial message delivery; Byzantine processes may lie ($N \geq 3f + 1$ is needed); in an asynchronous system, no deterministic algorithm can guarantee consensus with even one crash (**FLP result**).

E. LONG ANSWERS — 7/8/10 MARKS (complete)

E1. Explain system models: need, architectural models and fundamental models with diagrams.

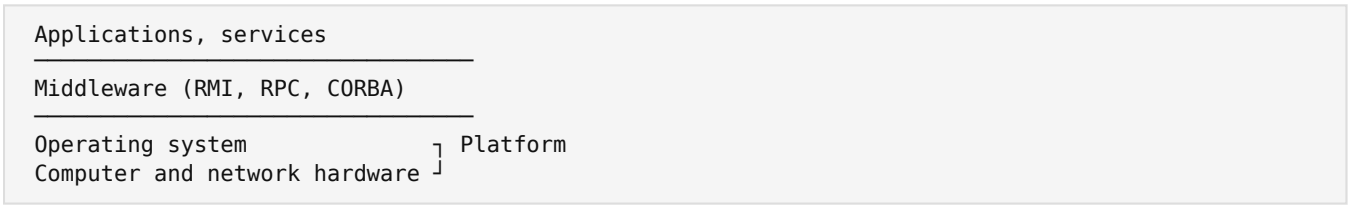
1. Introduction and definition A **system model** is an abstract, simplified description of the important aspects of a distributed system: its components, how they interact, and what assumptions are made about failures and security.

2. Need for system models 1. They **make assumptions explicit** (for example "messages may be lost", "clocks drift"). 2. They show the **common design problems** of all distributed systems. 3. They let us **reason about correctness** and prove whether an algorithm works (or is impossible). 4. They **generalise:** an algorithm proved for a model works for every real system that fits the model. 5. They help **compare** designs.



3. Architectural models These describe how the system is organised: its components, their placement on computers, and their roles.

(a) Software layers



The **platform** (hardware + OS) gives basic communication. **Middleware** hides heterogeneity and gives a common programming model.

(b) System architectures 1. **Client–server:** clients request, servers reply. `Client —request→ Server → (may call another server) Client ←reply—` 2. **Multiple servers:** a service is spread over several servers by **partitioning** (each server holds part of the data) or **replication** (each holds a copy). Examples: DNS, the web. 3. **Proxy servers and caches:** copies of recently used objects are kept near clients to reduce delay and load. `Client → Proxy/Cache → Web server` 4. **Peer processes:** all processes are equal and share resources (BitTorrent). `P1 ←→ P2 ▲ \ / ▲ ▼ / \ ▼ P4 ←→ P3` 5. **Mobile code:** code moves from server to client and runs there (JavaScript). 6. **Mobile agents:** a running program travels between computers, doing work. 7. **Thin clients:** the client only handles the display; the server does the processing.

(c) Design requirements: performance (responsiveness, throughput, load balancing), quality of service (reliability, security, adaptability), and use of caching and replication.

4. Fundamental models These are abstract models describing properties common to **all** distributed systems.

(i) Interaction model - Processes interact only by messages. - **Latency:** delay in delivering a message. **Bandwidth:** data carried per second. **Jitter:** variation in delivery time. - Each computer's clock has a **drift rate**, so there is no global time. - **Synchronous system:** known bounds on execution time, message delay and drift. **Asynchronous system:** no bounds (the Internet). - Event ordering is done by logical time (happened-before).

(ii) Failure model | Failure | Meaning | |---|---| | Omission (process) | Crash; fail-stop if it can be detected | | Omission (channel) | Message lost (send, channel or receive omission) | | Arbitrary (Byzantine) | Any wrong behaviour, including incorrect or malicious values | | Timing | Response outside the time limit (synchronous systems only) | - **Masking:** hide a failure (replication) or turn it into a milder kind (a checksum turns corruption into omission). - **Reliable communication = validity + integrity.**

(iii) Security model - Objects (resources) are protected by access rights; **principals** (users, processes) have authority. - **Enemy/attacker** can read, copy, alter or inject messages. - **Threats:** to processes (spoofing identity), to channels (eavesdropping, tampering, replay), denial of service, mobile code. - **Defences:** cryptography, authentication, secure channels (TLS/SSL), access control, digital signatures.

```
Process A ─── secure channel (encrypted, authenticated) ──► Process B
                  ▲ enemy tries to copy / change messages
```

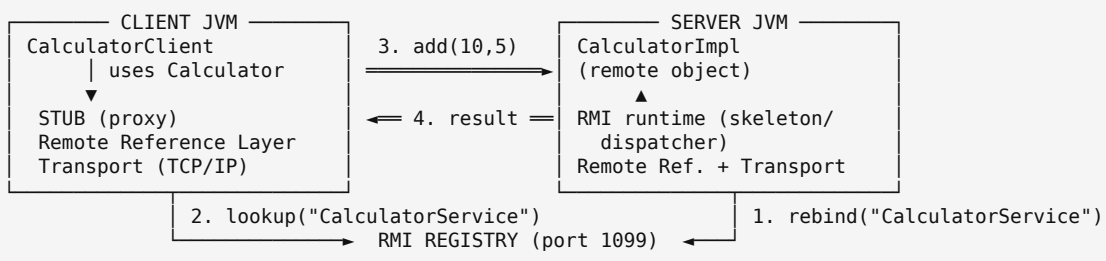
5. Comparison | Architectural model | Fundamental model | |---|---| | Structure and placement of components | Abstract behaviour and assumptions | | Client–server, P2P, proxy, mobile code | Interaction, failure, security | | "How is it built?" | "How does it behave and what can go wrong?" |

6. Conclusion: Architectural models tell us how to organise components; fundamental models tell us what assumptions our algorithms must handle. Together they guide the design of correct and efficient distributed systems.

E2. Design a simple Java RMI application (calculator): architecture, remote interface, server, client, registry, execution steps, code and output.

1. Introduction Java RMI (Remote Method Invocation) lets a Java program in one JVM call methods of an object located in another JVM, usually on another computer, as if the object were local. It uses **stubs** to hide communication, **serialisation** to pass objects, and provides **at-most-once** semantics.

2. Architecture



Layers: - **Stub/skeleton layer:** intercepts calls and marshals data. - **Remote reference layer:** manages references and invocation semantics. - **Transport layer:** TCP connections between the JVMs.

3. Components | Component | Description | |---|---| | Remote interface | `Calculator` extends `java.rmi.Remote`; every method declares `throws RemoteException` | | Remote object | `CalculatorImpl` implements `Calculator` and extends `UnicastRemoteObject` (this exports it so it can receive calls) | | Stub | Client-side proxy, generated automatically (Java 5+); marshals calls | | Skeleton | Server-side receiver; built into the RMI runtime since Java 1.2 | | RMI registry | Naming service on port 1099 that maps a name to a stub | | Server | Creates the object and registers it with `rebind` | | Client | Looks up the stub with `lookup` and calls methods |

4. Code

```
// ---------- Calculator.java (remote interface) ----------
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Calculator extends Remote {
    int add(int a, int b) throws RemoteException;
    int subtract(int a, int b) throws RemoteException;
    int multiply(int a, int b) throws RemoteException;
    double divide(int a, int b) throws RemoteException;
}

// ---------- CalculatorImpl.java (remote object) ----------
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class CalculatorImpl extends UnicastRemoteObject implements Calculator {
    public CalculatorImpl() throws RemoteException { super(); }
    public int add(int a, int b)       { return a + b; }
    public int subtract(int a, int b)  { return a - b; }
    public int multiply(int a, int b)  { return a * b; }
    public double divide(int a, int b) { return (double) a / b; }
}

// ---------- CalculatorServer.java ----------
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class CalculatorServer {
    public static void main(String[] args) {
        try {
            Calculator calc = new CalculatorImpl();                 // create remote object
            Registry registry = LocateRegistry.createRegistry(1099); // start registry
            registry.rebind("CalculatorService", calc);             // register name
            System.out.println("Calculator Server is ready...");
        } catch (Exception e) { e.printStackTrace(); }
    }
}

// ---------- CalculatorClient.java ----------
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class CalculatorClient {
    public static void main(String[] args) {
        try {
            Registry registry = LocateRegistry.getRegistry("localhost", 1099);
            Calculator calc = (Calculator) registry.lookup("CalculatorService");
            System.out.println("10 + 5 = " + calc.add(10, 5));
            System.out.println("10 - 5 = " + calc.subtract(10, 5));
            System.out.println("10 * 5 = " + calc.multiply(10, 5));
            System.out.println("10 / 5 = " + calc.divide(10, 5));
        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

5. Execution steps 1. Compile: `javac *.java`. 2. Start the server: `java CalculatorServer`. It creates the object, exports it (stub created), starts the registry and binds the name "CalculatorService". 3. Start the client: `java CalculatorClient`. It contacts the registry on port 1099 and looks up "CalculatorService", receiving the **stub**. 4. The client calls `calc.add(10, 5)` on the stub. 5. The stub serialises the method and arguments and sends them over TCP. 6. The server's RMI runtime unmarshals them and calls `add` on `CalculatorImpl`. 7. The result 15 is marshalled and returned; the stub unmarshals it and returns it to the client. 8. The client prints the result. The same happens for the other methods.

6. Sample output

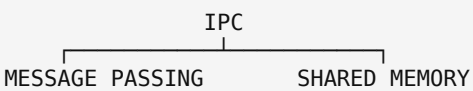
```
Server terminal:  Calculator Server is ready...
Client terminal:  10 + 5 = 15
                  10 - 5 = 5
                  10 * 5 = 50
                  10 / 5 = 2.0
```

7. Advantages and limitations | Advantages | Limitations | |---|---| | Object-oriented and simple | Works only Java-to-Java | | Stubs generated automatically | Slower than a local call | | Passes objects via serialisation | Firewall issues with dynamic ports | | Distributed garbage collection | Registry is very basic |

8. Conclusion: Java RMI hides network programming behind normal method calls, so a calculator on a server can be used from any client as if it were a local object.

E3. Explain interprocess communication in detail; compare message passing and shared memory.

1. Definition Interprocess Communication (IPC) is the set of mechanisms that let processes **exchange data and synchronise** their actions, either on the same computer or on different computers.



2. Message passing - Processes use `send(dest, msg)` and `receive(src, msg)`; no memory is shared. - **Synchronous (blocking):** the sender waits until the receiver takes the message. **Asynchronous (non-blocking):** the sender continues immediately. - **Direct communication:** to a named process. **Indirect:** through a mailbox or message queue. - **Sockets:** communication endpoint = IP address + port. - **UDP datagram communication:** connectionless, unreliable, messages may be lost or out of order; low overhead (DNS, streaming). - **TCP stream communication:** connection-oriented, reliable, ordered byte stream (web, e-mail, RMI). - **Marshalling and external data representation:** data is flattened to bytes in an agreed format (Java serialization, CORBA CDR, XML, JSON) so that heterogeneous machines understand it. - **Request–reply protocol:** doOperation (client), getRequest and sendReply (server). This is the basis of RPC and RMI.

```
Client                          Server
doOperation ── Request ──────► getRequest
  (wait)                        select & execute
           ◄────── Reply ─────── sendReply
```

3. Shared memory - Processes share a common memory area and communicate by reading and writing it. - Access must be synchronised with **locks, semaphores or monitors** to avoid race conditions. - On one computer: OS shared segments (`shmget`, `mmap`) or threads in one process. - In a distributed system: **Distributed Shared Memory (DSM)** gives the illusion of one shared address space; underneath, pages are moved by message passing.

```
P1 ──write──► [ SHARED MEMORY ] ◄──read── P2      (lock / semaphore)
```

4. Comparison | Basis | Message passing | Shared memory | |---|---|---| | Communication | Explicit send/receive | Read/write shared variables | | Where it works | Same or different machines | Same machine (or DSM) | | Speed | Slower | Faster | | Synchronisation | Implicit (blocking calls) | Explicit locks/semaphores | | Data copying | Yes | No | | Programming burden | Message formats, failures | Race conditions, deadlocks | | Scalability | High | Low | | Security/isolation | Good | Weak | | Failure effect | Lost messages | Corrupted shared data | | Examples | Sockets, MPI, RPC/RMI, Kafka | Threads, POSIX shm, DSM |

5. Advantages and limitations - **Message passing:** + works across networks, + isolation, + scalability; − slower, − more programming for messages. - **Shared memory:** + very fast, + natural programming; − limited to one machine, − needs careful synchronisation, − DSM has coherence overhead.

6. Applications - **Message passing:** client–server systems, chat, distributed databases, cluster computing (MPI). - **Shared memory:** multithreaded servers, producer–consumer buffers, multiprocessor scientific programs.

7. Conclusion: Since computers in a distributed system do not share physical memory, **message passing** is the basic IPC method; shared memory is used inside a single machine or emulated through DSM.

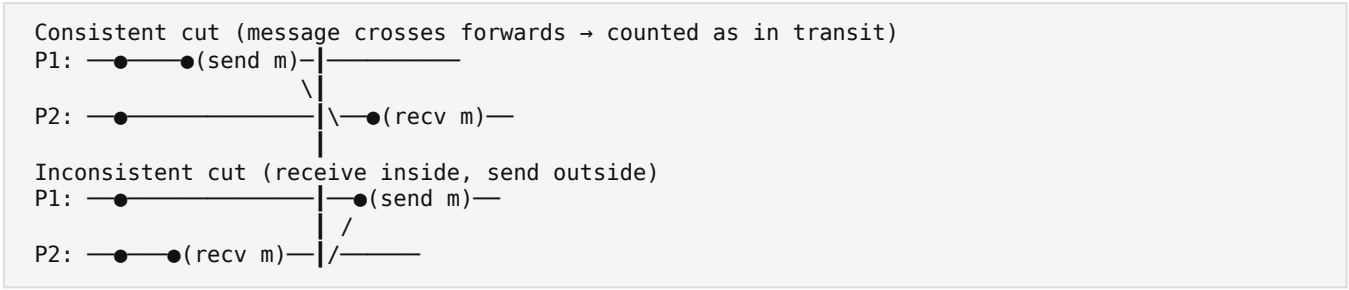
E4. Explain global states, consistent cut and the distributed snapshot algorithm with a P1–P2–P3 example.

1. Global state The **global state** is the collection of the **local states of all processes** plus the **states of all channels** (messages in transit) at an instant. - Local state Si = the values of the variables of process Pi. - Channel state Cij = messages sent by Pi but not yet received by Pj.

2. Why a consistent global state is important 1. **Deadlock detection:** find cycles of waiting processes. 2. **Termination detection:** check that all processes are idle and no messages are in transit. 3. **Distributed garbage collection:** find objects no longer referenced. 4. **Checkpointing and recovery:** restart from a saved consistent state after a failure. 5. **Debugging and auditing:** for example, check that total money is preserved.

Problem: there is no global clock, so processes cannot record their states at exactly the same moment.

3. Cut and consistency - A **cut** is a set of events of the processes, drawn as a line across the timelines (a prefix of each process's history). - A cut is **consistent** if for every event in the cut, all events that happened before it are also in the cut. In practice: **if a receive event is in the cut, its send event must also be in the cut.** - An **inconsistent cut** contains a message that is received but never sent (an **orphan message**), which is impossible in reality.



4. Chandy–Lamport snapshot algorithm Assumptions: FIFO, reliable, one-directional channels; strongly connected processes; no failures during the snapshot; a MARKER control message is used.

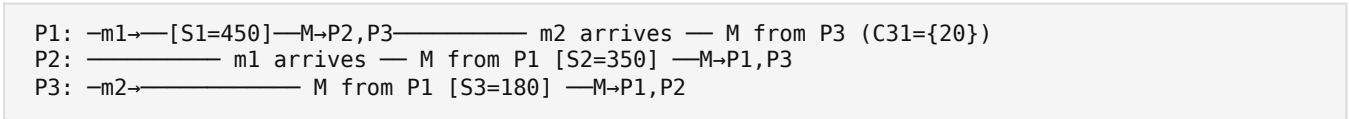
Marker sending rule (process P): record P's state, then send a marker on every outgoing channel before sending any other message.

Marker receiving rule (process Q, on channel c): - **If Q has not recorded its state:** record the state, set the state of c = ∅, follow the marker sending rule, and start recording messages on all other incoming channels. - **Else:** the state of c = messages received on c since Q recorded its state; stop recording c.

The algorithm terminates when every process has received a marker on every incoming channel.

5. Example (P1, P2, P3 bank) Initial balances: P1 = ₹500, P2 = ₹300, P3 = ₹200 (total ₹1000).

Step	Event	Effect
1	P1 sends ₹50 to P2 (m1)	P1 = 450
2	P1 starts snapshot	Records **S1 = 450**; markers on C12, C13; records C21, C31
3	P3 sends ₹20 to P1 (m2)	P3 = 180
4	P2 receives m1	P2 = 350
5	P2 receives marker on C12	Records **S2 = 350**, C12 = ∅; markers on C21, C23
6	P3 receives marker on C13	Records **S3 = 180**, C13 = ∅; markers on C31, C32
7	P1 receives m2 (recording C31), then marker on C31	**C31 = {₹20}**
8	Remaining markers arrive	C21 = C23 = C32 = ∅



Recorded global state: S1 = 450, S2 = 350, S3 = 180, C31 = {₹20}. Total = 450 + 350 + 180 + 20 = **₹1000** ✓. The money is conserved, so the snapshot is consistent.

6. Properties - **Messages:** one marker per channel, O(E); for full connectivity N(N−1). - The application **keeps running** during the snapshot. - The recorded state is consistent and reachable, though it may not have existed at one real instant.

7. Conclusion: The Chandy–Lamport algorithm records a consistent global state using markers, and it is the basis for detecting stable properties such as deadlock and termination.

E5. Explain distributed mutual exclusion: requirements and four algorithms with message complexity.

1. Introduction Distributed mutual exclusion ensures that only **one process at a time** executes its **critical section (CS)** to access a shared resource, using only messages (no shared variables or semaphores). *Examples:* updating a shared file or bank record, using a shared printer.

2. Requirements - **ME1 Safety:** at most one process is in the CS at any time. - **ME2 Liveness:** requests to enter and exit the CS eventually succeed (no deadlock, no starvation). - **ME3 Ordering:** if one request happened-before another, it is granted first (fairness).

Performance measures: messages per entry (bandwidth), client delay, and synchronisation delay (time between one process exiting and the next entering).

3. Central server algorithm

```
P ──REQUEST──► Server ──GRANT──► P  ...CS...  P ──RELEASE──► Server ──GRANT──► next
```

- The server keeps a token and a queue. If the token is free it grants immediately; otherwise it queues the request.
- **Messages: 3** per entry (request, grant, release). Sync delay: 2 messages.
- **Pros:** simple and fair. **Cons:** single point of failure, bottleneck.

4. Ring-based (token ring) algorithm

```
P1 → P2 → P3 → P4 → P1   (the token circulates; only its holder may enter the CS)
```

- A process waits for the token, enters the CS, then passes the token on.
- **Messages:** 1 to N−1 per entry, and the token keeps circulating even when no one needs it.
- **Pros:** no starvation, simple. **Cons:** token loss, a crash breaks the ring, wastes bandwidth.

5. Ricart–Agrawala algorithm - The requester multicasts REQUEST(T, id) and waits for N−1 REPLIES. - A receiver replies immediately unless it is in the CS, or wants the CS and has an earlier timestamp; in that case it defers the reply. - On exit, the process replies to all deferred requests. - **Messages: 2(N−1)**. Sync delay: 1 message. - **Pros:** fully distributed and fair. **Cons:** many messages; any crash blocks the others.

6. Lamport's mutual exclusion algorithm - Each process keeps a queue of requests ordered by timestamp. - The requester sends REQUEST to all; each receiver adds it to its

queue and replies. - A process enters when its request is at the head of its own queue and it has received a later-timestamped message from every other process. - On exit it sends RELEASE to all. - **Messages: 3(N−1)**. Needs FIFO channels.

7. (Extra) Maekawa's voting algorithm: a process asks only its voting set (about √N processes, and any two sets overlap). About **3√N** messages; can deadlock.

8. Comparison (N processes) | Algorithm | Messages per entry | Sync delay | Main problem | |---|---|---|---| | Central server | 3 | 2 | Server crash | | Token ring | 1 … N−1 | 1 … N−1 | Token loss | | Ricart–Agrawala | 2(N−1) | 1 | Any crash blocks | | Lamport | 3(N−1) | 1 | Heavy, needs FIFO | | Maekawa | ≈3√N | 2 | Deadlock |

Example with N = 5: central = 3, RA = 8, Lamport = 12.

9. Conclusion: A central server uses the fewest messages but is a single point of failure; token and permission-based algorithms remove the central server at the cost of more messages. No algorithm tolerates process crashes without extra mechanisms.

E6. Explain election algorithms: Bully and Ring, with examples and comparison.

1. Need for election Many algorithms need a **coordinator** (central-server mutual exclusion, the Berkeley time master, the primary replica). When the coordinator crashes, the remaining processes must choose a new one. An election algorithm chooses one process, usually the alive process with the **highest ID**, and makes all processes agree on it.

Requirements: E1 Safety: every participating process selects the same process (the one with the largest ID). **E2 Liveness:** all processes eventually participate and decide, or crash.

2. Bully algorithm *Assumptions:* synchronous system (reliable timeouts), all IDs known, reliable messages. *Messages:* ELECTION, OK, COORDINATOR. *Steps:* 1. The detecting process sends ELECTION to all higher-ID processes. 2. If there is no reply, it becomes coordinator and sends COORDINATOR to all. 3. If some process replies OK, it waits; each higher process repeats step 1. 4. The highest alive process wins.

Example (P1–P5, P5 crashes, P2 detects):

```
P2 —ELECTION→ P3, P4, P5✗      P3, P4 —OK→ P2
P3 —ELECTION→ P4, P5✗          P4 —OK→ P3
P4 —ELECTION→ P5✗  (timeout)
P4 —COORDINATOR→ P1, P2, P3    ⇒ P4 is the coordinator
```

Messages: best **N−2**, worst **O(N²)**.

3. Ring algorithm (Chang–Roberts) *Assumptions:* logical one-directional ring; each process knows its successor (crashed processes are skipped); works in asynchronous systems. *Steps:* 1. The starter marks itself a participant and sends ELECTION(own id) to its successor. 2. A receiver compares the id: - If the received id is larger: forward it. - If it is smaller and the receiver is not yet a participant: replace it with its own id and forward. - If it is smaller and the receiver is already a participant: discard it. - If it equals the receiver's own id: the receiver has won. 3. The winner sends ELECTED(id) around the ring; every process records the coordinator.

Example (ring P1→P2→P3→P4→P5→P1, P5 crashed so P4→P1; P2 starts):

```
P2: ELECTION(2) → P3: replace → ELECTION(3) → P4: replace → ELECTION(4)
→ P1: forward 4 → P2: forward 4 → P3: forward 4 → P4: own id ⇒ WINNER
P4: ELECTED(4) → P1 → P2 → P3 → P4 (stop)
```

Messages: best **2N**, worst **3N − 1**.

4. Comparison | Feature | Bully | Ring | |---|---|---| | Topology | Any process can reach any other | Logical ring | | Knowledge needed | All IDs | Only the successor | | System model | Synchronous (timeouts) | Asynchronous is fine | | Messages | ELECTION, OK, COORDINATOR | ELECTION, ELECTED | | Best case | N − 2 | 2N | | Worst case | O(N²) | 3N − 1 | | Winner | Highest alive ID | Highest alive ID | | Weakness | Many messages, depends on timeouts | Ring breaks if a process fails during the election; slow |

5. Conclusion: Both elect the highest-ID process. Bully is fast when a high-ID process detects the failure but costs O(N²) messages in the worst case; Ring uses O(N) messages and weaker assumptions but is slower and needs ring maintenance.

E7. Explain consensus in distributed systems: requirements, process failures, communication delays, Byzantine generals and related problems.

1. Definition and need Consensus: every process proposes a value, and all correct processes must **agree on one value**. *Uses:* commit or abort a transaction, keep replicas consistent (Paxos, Raft), elect a leader, total-order multicast, blockchain.

2. Requirements - **Termination:** every correct process eventually decides. - **Agreement:** all correct processes decide the same value. - **Validity/Integrity:** if all correct processes propose v, the decision is v (the decided value was proposed).

3. Example: five replicas vote on commit (1) or abort (0): P1 = 1, P2 = 1, P3 = 1, P4 = 0, P5 = 1. After exchanging votes, all apply the rule "majority", so all decide **1 (commit)**.

4. Consensus with crash failures (synchronous system)

```
Values_i = {v_i}
for round r = 1 .. f+1:
    B-multicast the values not sent before
    add all values received this round to Values_i
decide d_i = min(Values_i)        (or majority)
```

Why f + 1 rounds: at most f processes crash, so at least one round has no crash; after that round all correct processes hold the same set.

Case: 5 processes, at most 1 crash (f = 1) → 2 rounds. 1. Round 1: all send their values; P5 crashes after sending only to P1 and P2. 2. Round 2: P1 and P2 forward P5's value to P3 and P4. 3. Now P1–P4 hold identical sets and apply the same rule, so they agree.

5. Challenges from process failures - A **crash** during a multicast means only some processes get the value. - **Byzantine** processes send different values to different processes. - A process that recovers may have missed messages.

6. Challenges from communication delays - In an **asynchronous** system a slow message looks like a crashed process. - Timeouts may wrongly declare a process dead, or waiting may never end. - **FLP impossibility (1985):** in an asynchronous system, no deterministic algorithm can guarantee consensus if even one process can crash. - Practical solutions: timeouts (partial synchrony), failure detectors, randomisation, and quorum-based protocols such as **Paxos and Raft** (2f + 1 nodes tolerate f crashes).

7. Byzantine Generals Problem A commander sends an order ("attack" or "retreat") to lieutenants; some generals may be traitors. - **BG1 Agreement:** all loyal lieutenants follow the same order. - **BG2 Integrity:** if the commander is loyal, the loyal lieutenants follow his order. - **Result:** with oral (unsigned) messages it is solvable only if **N ≥ 3f + 1**, and f + 1 rounds are needed.

Example: N = 3, f = 1 is impossible. A loyal lieutenant receiving conflicting reports cannot tell whether the commander or the other lieutenant is lying. *Example: N = 4, f = 1 works:*

```
Commander C (loyal) sends "attack" to L1, L2, L3. L3 is a traitor.
L1 hears: C = attack, L2 relays attack, L3 relays retreat → majority = attack
L2 hears: C = attack, L1 relays attack, L3 relays retreat → majority = attack   ✓ agreement
```

8. Related problems | Problem | Meaning | |---|---| | Interactive consistency | All agree on a **vector** of values, one per process | | Byzantine generals | One commander's value must be agreed despite traitors | | Total-order multicast | All deliver messages in the same order (equivalent to consensus) | | Atomic commit (2PC) | All commit or all abort | | Leader election | Agree on one leader | Consensus, Byzantine generals and interactive consistency can each be solved using any of the others, so they are **equivalent** problems.

9. Conclusion: Consensus is the heart of fault-tolerant distributed computing. It needs f + 1 rounds against crashes in synchronous systems and N ≥ 3f + 1 against Byzantine faults, and it cannot be guaranteed in purely asynchronous systems (FLP).

E8. Explain synchronisation: physical clock algorithms (Cristian, Berkeley, NTP) and logical clocks.

1. Introduction Distributed systems have **no global clock**. Synchronisation means making processes agree on **time** (physical clocks) or on the **order of events** (logical clocks).

2. Physical clocks: basic terms - **Skew:** the difference between two clocks at the same instant. - **Drift:** the rate at which a clock deviates from real time ($\rho \approx 10^{-6}$). - **UTC:** the international standard time. - **External synchronisation:** clocks are kept close to UTC. **Internal synchronisation:** clocks are kept close to each other. - Resynchronise every $\delta / 2\rho$ seconds to keep skew $\leq \delta$.

3. Cristian's algorithm (external)

```
Client T0 ──request──► Time server
Client T1 ◄──Ts──────── Time server
new time = Ts + (T1 − T0)/2
```

Example: T0 = 0 ms, T1 = 20 ms, Ts = 10:00:00.500 → new time = 10:00:00.510. It is simple, but the server is a single point of failure.

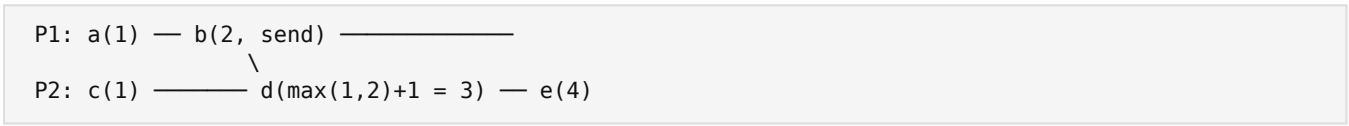
4. Berkeley algorithm (internal) 1. The master polls every slave for its time. 2. It estimates each slave's time (compensating for RTT) and averages them (ignoring faulty values). 3. It sends each machine the **adjustment**, not the absolute time.

Example: master 3:00, slaves 2:50 and 3:25. Offsets 0, −10, +25; average = +5 → target 3:05. Adjustments: master +5, slave 1 +15, slave 2 −20. It needs no UTC source, but the master can fail (a new one is then elected).

5. NTP (Network Time Protocol) - A hierarchy of **strata:** stratum 0 = atomic/GPS clocks, stratum 1 = servers attached to them, stratum 2 syncs from stratum 1, and so on. - Modes: multicast, procedure-call, symmetric. - Offset **θ = ((t1 − t0) + (t2 − t3))/2**; delay **δ = (t3 − t0) − (t2 − t1)**. The sample with the lowest delay is chosen. - Scalable, fault-tolerant and secure; accuracy is tens of milliseconds over the Internet.

6. Logical clocks Happened-before: same process order, send → receive, transitivity. **Lamport clock rules:** increment before each event; send the timestamp; on receive L = max(L, t) + 1. Guarantees a → b ⇒ L(a) < L(b).

Example:



Vector clocks: an array of N counters; on receive take the component-wise maximum and then increment own entry. a → b ⇔ V(a) < V(b), so vector clocks can detect concurrency.

7. Logical vs physical | Physical | Logical | |---|---| | Real time | Order of events only | | Needs synchronisation | No synchronisation needed | | Affected by drift | Not affected | | Cristian, Berkeley, NTP | Lamport, vector clocks |

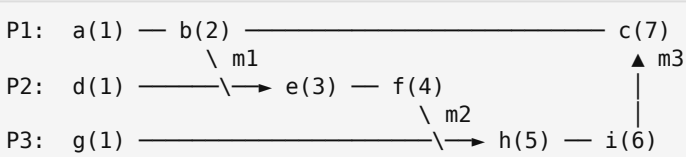
8. Conclusion: Physical clock synchronisation keeps real time approximately correct, while logical clocks give an exact causal order of events. Distributed systems use both depending on the need.

F. NUMERICAL / PROBLEM-BASED ANSWERS (fully worked)

F1. Lamport timestamps for P1 (a, b = send m1, c = recv m3), P2 (d, e = recv m1, f = send m2), P3 (g, h = recv m2, i = send m3).

Rules: increment before each event; attach the clock value when sending; on receive L = max(L, t) + 1.

Event	Process	Type	Working	Timestamp
a	P1	internal	0 + 1	**1**
b	P1	send m1	1 + 1 (m1 carries 2)	**2**
d	P2	internal	0 + 1	**1**
e	P2	receive m1 (t = 2)	max(1, 2) + 1	**3**
f	P2	send m2	3 + 1 (m2 carries 4)	**4**
g	P3	internal	0 + 1	**1**
h	P3	receive m2 (t = 4)	max(1, 4) + 1	**5**
i	P3	send m3	5 + 1 (m3 carries 6)	**6**
c	P1	receive m3 (t = 6)	max(2, 6) + 1	**7**



Ans: a = 1, b = 2, c = 7, d = 1, e = 3, f = 4, g = 1, h = 5, i = 6.

F2. Clock 14 receives timestamp 20; clock 30 receives timestamp 25.

- Case 1: max(14, 20) + 1 = **21**.
- Case 2: max(30, 25) + 1 = **31**. The local clock only moves forward; it never goes back even when the message timestamp is smaller.

F3. Cristian: T0 = 12:00:00.000, T1 = 12:00:00.600, Ts = 12:00:10.200.

- RTT = T1 − T0 = 0.600 s.
- New time = Ts + RTT/2 = 12:00:10.200 + 0.300 = **12:00:10.500**.
- Accuracy = ±(RTT/2 − min) = **±0.3 s** (taking min ≈ 0).

F4. Berkeley: master 2:00, slaves S1 = 1:50, S2 = 2:05, S3 = 2:20 (master included).

Step 1 – offsets from the master: master 0, S1 −10 min, S2 +5 min, S3 +20 min. **Step 2 – average:** (0 − 10 + 5 + 20) / 4 = 15 / 4 = **+3.75 min** (3 min 45 s). **Step 3 – target time:** 2:00 + 3:45 = **2:03:45**. **Step 4 – adjustment = average − own offset:** | Machine | Offset | Adjustment | |---|---|---| | Master | 0 | **+3 min 45 s** | | S1 | −10 min | 3.75 − (−10) = **+13 min 45 s** | | S2 | +5 min | 3.75 − 5 = **−1 min 15 s** | | S3 | +20 min | 3.75 − 20 = **−16 min 15 s** |

All clocks become 2:03:45. Negative adjustments are applied by **slowing the clock down**, not setting it back. The master may also treat S3 (20 min off) as a faulty outlier and exclude it from the average.

F5. $\rho = 2 \times 10^{-6}$, allowed skew $\delta = 2$ ms.

- Two clocks drift apart by up to 2ρ per second = 4×10^{-6} s per second.
- Resync interval $\leq \delta / (2\rho) = 0.002 / (4 \times 10^{-6})$ = **500 s** (about 8.3 minutes).
- Drift of one clock in a day = $\rho \times 86\,400 = 2 \times 10^{-6} \times 86\,400$ = **0.1728 s ≈ 173 ms**.

F6. N = 8 processes.

- Ricart–Agrawala = $2(N - 1) = 2 \times 7$ = **14**.
- Lamport = $3(N - 1) = 3 \times 7$ = **21**.
- Central server = **3** (request, grant, release).

F7. Bully: P1–P6, P6 fails, P3 detects.

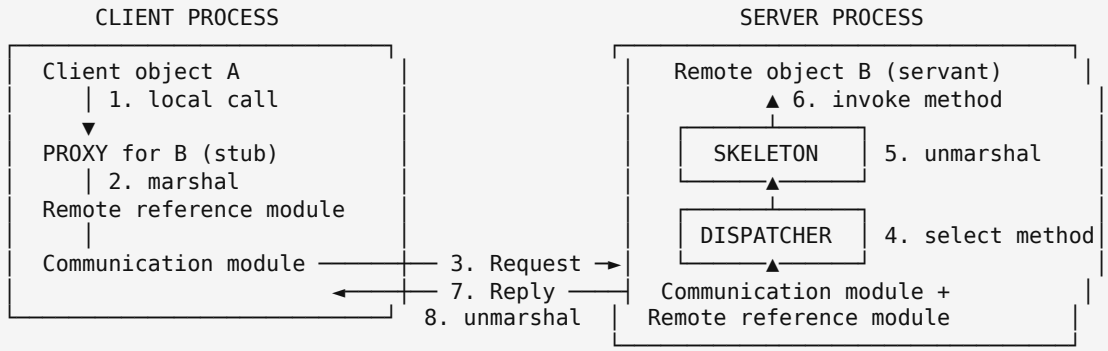
Step	Messages	Count
P3 → ELECTION to P4, P5, P6	3	3
P4, P5 → OK to P3 (P6 silent)	2	5
P4 → ELECTION to P5, P6	2	7
P5 → OK to P4	1	8
P5 → ELECTION to P6 (timeout)	1	9
P5 → COORDINATOR to P1, P2, P3, P4	4	13
Winner: P5 (highest alive ID). **Total = 13 messages.**		

F8. Ring P1→P4→P2→P6→P3→P1; P6 (the coordinator, highest ID) has failed; P2 starts.

Alive ring: P1 → P4 → P2 → P3 → P1 (P6 is skipped). | # | From → To | Message | Reason | |---|---|---|---| | 1 | P2 → P3 | ELECTION(2) | P2 starts | | 2 | P3 → P1 | ELECTION(3) | 3 > 2, P3 not participant → replaces with own id | | 3 | P1 → P4 | ELECTION(3) | 1 < 3 → forward | | 4 | P4 → P2 | ELECTION(4) | 4 > 3 → replaces with own id | | 5 | P2 → P3 | ELECTION(4) | forward | | 6 | P3 → P1 | ELECTION(4) | forward | | 7 | P1 → P4 | ELECTION(4) | forward | | – | P4 | — | receives its own id ⇒ **P4 is the coordinator** | | 8–11 | P4 → P2 → P3 → P1 → P4 | ELECTED(4) | announcement circulates once | **Winner: P4. Total = 7 election + 4 elected = 11 messages.**

G. DIAGRAM-BASED ANSWERS (diagram + explanation)

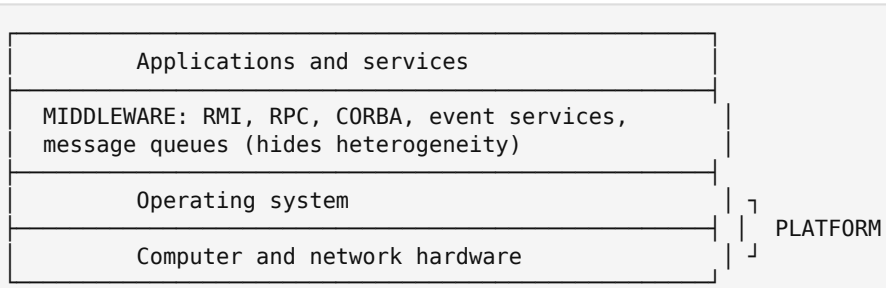
G1. RMI architecture (proxy, dispatcher, skeleton, communication module).



Explanation: - **Proxy (stub):** makes the remote call look local; marshals the request and unmarshals the reply. - **Remote reference module:** translates between local and remote object references using the remote object table. - **Communication module:** carries out the request–reply protocol. - **Dispatcher:** receives the request and selects the right method using the method id. - **Skeleton:** unmarshals the arguments, calls the servant, and marshals the result. - **Servant:** the real object that does the work.

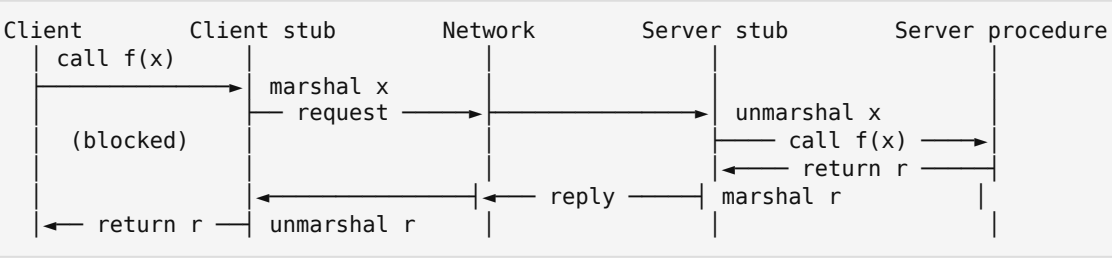
The numbers 1–8 in the diagram give the order of a complete invocation.

G2. Software layers / middleware architecture.



Explanation: - **Platform:** the lowest layers (hardware + OS), for example x86 + Linux. They provide basic communication. - **Middleware:** masks differences in OS, hardware, network and programming language, and gives a uniform model (remote invocation, events, groups). - **Applications:** use middleware services without worrying about distribution. - Each layer uses only the services of the layer below it.

G3. Message sequence for an RPC call.



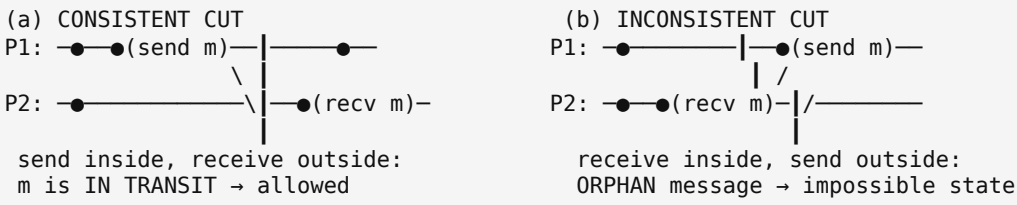
Explanation: the client is blocked while waiting; the stubs handle marshalling; the reply returns along the same path. If the reply is lost, the client stub retransmits according to the invocation semantics.

G4. Timeline showing happened-before and concurrent events (3 processes).



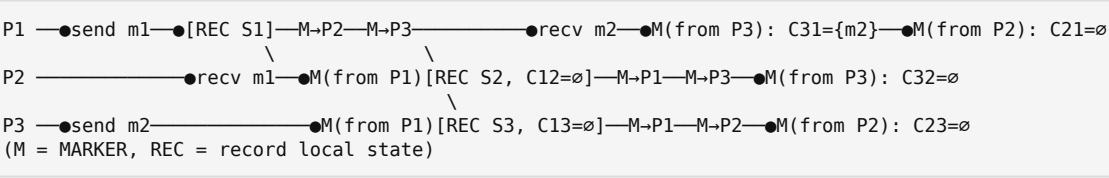
Happened-before: a → b, b → d, d → e, e → h; so a → h and b → h. **Concurrent pairs:** a ‖ c, a ‖ g, f ‖ d, f ‖ e, f ‖ h, c ‖ b. **Rule:** two events are concurrent if no chain of process-order and message arrows connects them.

G5. Consistent and inconsistent cut.



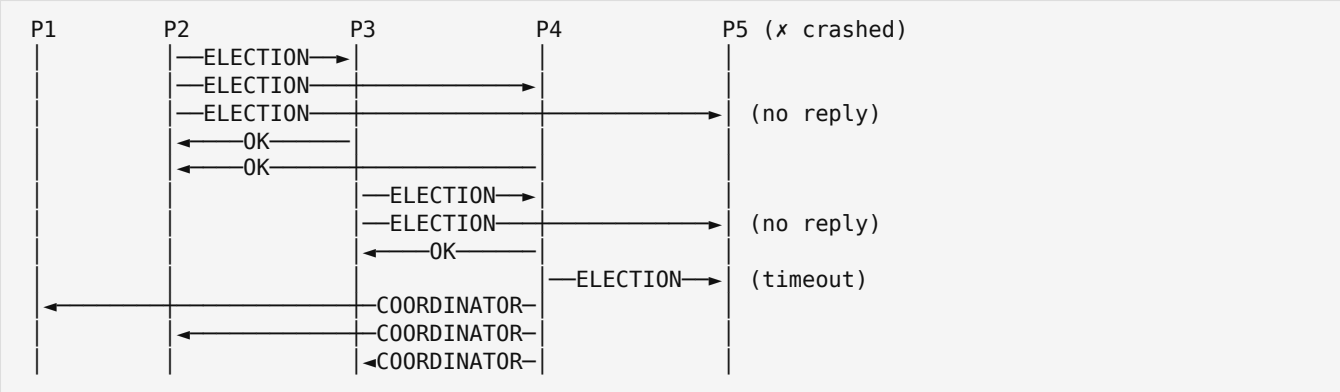
Explanation: - A cut is consistent if, for every receive event inside it, the matching send is also inside. - In (a), m is counted in the channel state. - In (b), P2 has received a message that P1 has "not yet sent", so the state could never have occurred.

G6. Chandy–Lamport snapshot message flow for P1, P2, P3.



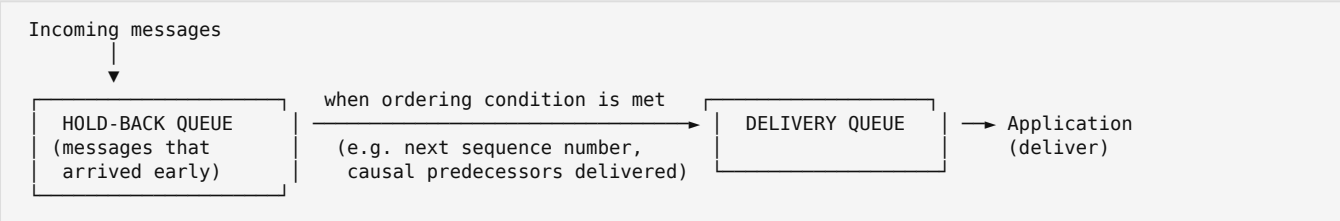
Explanation: P1 initiates; every process records its state on the first marker and forwards markers; messages that arrive after recording but before a channel's marker (m2 on C31) become that channel's state. With the bank example: 450 + 350 + 180 + 20 = ₹1000.

G7. Bully election message flow with 5 processes.



Explanation: P2 detects the failure; higher processes reply OK and take over; P4, the highest alive, gets no reply from P5 and announces itself.

G8. Hold-back queue for ordered multicast.



Explanation: - A received message is not given to the application immediately. It waits in the hold-back queue until the ordering guarantee is met. - **FIFO:** the next sequence number from that sender has arrived. - **Causal:** all causally earlier messages (vector timestamps) have been delivered. - **Total:** the message's agreed sequence number is next. - Then it moves to the delivery queue and is delivered.

H. COMPARE / DIFFERENTIATE ANSWERS (full tables)

H1. Client–server vs peer-to-peer

Basis	Client–server	Peer-to-peer
Roles	Fixed: clients request, server serves	Every peer is both client and server
Control	Centralised	Decentralised
Single point of failure	Yes (server)	No
Scalability	Limited by server capacity	Improves as peers join
Cost	Expensive servers	Low (uses peers' resources)
Security and management	Easy, central	Difficult
Data availability	Depends on server	Depends on peers being online
Examples	Web, e-mail, banking	BitTorrent, blockchain

H2. Message passing vs shared memory

Basis	Message passing	Shared memory
Method	send/receive	Read/write common memory
Works across network	Yes	No (only via DSM)
Speed	Slower	Faster
Synchronisation	Implicit	Explicit locks/semaphores
Scalability	High	Low
Isolation	Good	Poor
Problems	Lost messages, formats	Race conditions
Examples	Sockets, MPI, RMI	Threads, shm, DSM

H3. RMI vs RPC

Basis	RPC	RMI
Paradigm	Procedural	Object-oriented
Invokes	Remote procedure	Method of a remote object
Parameters	Plain data, by value	Objects and remote references
Language	Language-neutral (IDL)	Java only (Java RMI)
Stubs	Client and server stubs	Stub and skeleton
Naming	Portmapper/binder	RMI registry
Garbage collection	No	Distributed GC
Overhead	Lower	Higher
Example	Sun RPC, gRPC	Java RMI

H4. Synchronous vs asynchronous distributed system

Basis	Synchronous	Asynchronous
Process execution time	Known bound	No bound
Message delay	Known bound	No bound
Clock drift	Known bound	No bound
Failure detection	Reliable with timeouts	Not reliable
Algorithms	Easier (e.g. f + 1 round consensus)	Harder (FLP impossibility)
Examples	Real-time embedded/control systems	The Internet

H5. Logical vs physical time

Basis	Physical time	Logical time
Meaning	Real time (UTC)	Order of events
Source	Hardware/atomic clocks	Counters updated on events
Synchronisation needed	Yes	No
Drift and skew	Affected	Not affected
Answers	"What time is it?"	"Which event came first?"
Overhead	Periodic sync messages	Timestamp in each message
Algorithms	Cristian, Berkeley, NTP	Lamport, vector clocks
Uses	Timestamps, expiry, scheduling	Mutual exclusion, causal ordering, debugging

H6. Lamport vs vector clocks

Basis	Lamport clock	Vector clock
Size	One integer	Array of N integers
Receive rule	L = max(L, t) + 1	V[j] = max(V[j], t[j]) for all j, then V[i]++
Property	$a \rightarrow b \Rightarrow L(a) < L(b)$	$a \rightarrow b \Leftrightarrow V(a) < V(b)$
Detects concurrency	No	Yes
Message overhead	Small (O(1))	Larger (O(N))
Total ordering	Easy (add process id)	Not directly
Uses	Mutual exclusion, total ordering	Causal multicast, debugging, conflict detection

H7. Bully vs Ring

Basis	Bully	Ring
Topology	Any-to-any	Logical ring
Knowledge	All IDs	Successor only

Basis	Bully	Ring
Timing	Synchronous (timeouts)	Asynchronous is fine
Messages	ELECTION, OK, COORDINATOR	ELECTION, ELECTED
Best case	$N - 2$	$2N$
Worst case	$O(N^2)$	$3N - 1$
Winner	Highest alive ID	Highest alive ID
Weakness	Message storm, timeout errors	Ring break, slow

H8. Cristian vs Berkeley

Basis	Cristian	Berkeley
Type	External (to UTC)	Internal (within group)
Server role	Passive time server answers	Active master polls slaves
Result	Client sets Ts + RTT/2	Everyone gets an adjustment to the average
UTC receiver needed	Yes	No
Faulty clocks	Server fault affects all	Outliers ignored
Failure	Server is a single point of failure	Master failure → elect new master
Suited for	Clients syncing with a time server	LAN group of machines

H9. Token-based vs non-token-based mutual exclusion

Basis	Token-based	Non-token (permission-based)
Entry rule	Process holding the unique token enters	Process collects permission (replies)
Examples	Token ring, Suzuki–Kasami	Ricart–Agrawala, Lamport, Maekawa
Messages	1 to N	$2(N-1)$, $3(N-1)$, $\approx 3\sqrt{N}$
Safety comes from	Only one token exists	Timestamp ordering / voting
Main problem	Token loss, regeneration	A crashed process never replies
Starvation	Avoided by circulation order	Avoided by timestamps

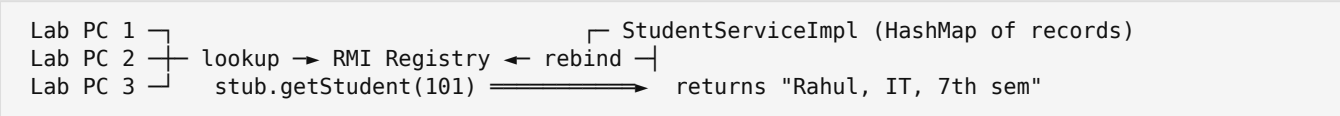
H10. Crash vs Byzantine failures

Basis	Crash failure	Byzantine failure
Behaviour	Process stops	Arbitrary, possibly malicious
Wrong values sent	No	Yes, possibly different to different processes
Detection	Timeouts (synchronous)	Very hard
Processes needed to tolerate f	$N \geq f + 1$ (sync flooding), $2f + 1$ (quorum protocols)	$N \geq 3f + 1$
Rounds (synchronous)	$f + 1$	$f + 1$
Example	Power failure	Hacked node, memory corruption

I. CASE-STUDY ANSWERS (complete)

I1. Student Information Service using Java RMI.

Design: a central server holds student records; lab computers (clients) call `getStudent(roll)` remotely.



Code:

```
// StudentService.java
import java.rmi.*;
public interface StudentService extends Remote {
    String getStudent(int roll) throws RemoteException;
}
// StudentServiceImpl.java
import java.rmi.*; import java.rmi.server.*; import java.util.*;
public class StudentServiceImpl extends UnicastRemoteObject implements StudentService {
    private final Map<Integer, String> db = new HashMap<>();
    public StudentServiceImpl() throws RemoteException {
        db.put(101, "Rahul, IT, Sem VII");
        db.put(102, "Priya, IT, Sem VII");
    }
    public String getStudent(int roll) { return db.getOrDefault(roll, "Not found"); }
}
// StudentServer.java
import java.rmi.registry.*;
public class StudentServer {
    public static void main(String[] a) throws Exception {
        LocateRegistry.createRegistry(1099).rebind("StudentService", new StudentServiceImpl());
        System.out.println("Student server ready");
    }
}
// StudentClient.java
import java.rmi.registry.*;
public class StudentClient {
    public static void main(String[] a) throws Exception {
        StudentService s = (StudentService) LocateRegistry.getRegistry("localhost", 1099)
                                                           .lookup("StudentService");
        System.out.println(s.getStudent(101));
    }
}
```

Output: `Student server ready` (server) and `Rahul, IT, Sem VII` (client). **Execution steps:** start the server (registry + rebind) → the client looks up and gets the stub → it calls `getStudent` → the stub marshals the call → the server executes it → the result is returned. **Benefits:** location transparency; one central copy of the data; clients stay simple. **Improvement:** replicate the server or add a database for reliability.

I2. Consistent total of three bank branches (P1 = 500, P2 = 300, P3 = 200).

Problem: transfers are in progress; reading each branch at a different time may count money twice or miss money in transit. **Solution:** use the **Chandy–Lamport snapshot**: 1. P1 records its balance and sends MARKERs to P2 and P3, then records its incoming channels. 2. Each branch records its balance on its first marker, sends markers, and records its other incoming channels. 3. Money arriving on a channel after recording but before that channel's marker is recorded as **in transit**. 4. Total = Σ recorded balances + Σ in-transit amounts. **Example:** P1

sends ₹50 to P2 before the snapshot; P3 sends ₹20 to P1 during it. Recorded: P1 = 450, P2 = 350, P3 = 180, C31 = 20 → **₹1000** ✓. **Requirements:** FIFO, reliable channels. The banking application keeps running during the snapshot.

I3. Five replicated database servers; one may crash. Agree on commit or abort.

Approach (synchronous, f = 1 crash) – flooding consensus with f + 1 = 2 rounds: 1. Each server proposes its vote (commit or abort) and multicasts it to the others. 2. Round 1: suppose S5 crashes after sending its vote to S1 and S2 only. 3. Round 2: every server multicasts any votes it learned in round 1, so S3 and S4 get S5's vote. 4. All correct servers (S1–S4) now hold the same set of votes. 5. Decision rule (same for all): commit only if all votes are commit, otherwise abort (atomic commit rule), or use a majority. 6. **Agreement:** same set + same rule → same decision. **Validity:** the decision is one of the proposed votes. **Termination:** decided after 2 rounds.

Challenges: - **Crash failures:** partial multicast leads to different views, which is why the extra round is needed. - **Communication delays:** in an asynchronous network a slow server looks crashed; wrong timeouts break agreement (FLP). - **Message overhead:** $O(N^2)$ messages per round. - **Byzantine case:** if the faulty server can lie, $N \geq 3f + 1 = 4$ is needed (5 ✓), and majority voting on relayed values is used.

Practical systems: Paxos or Raft (a majority of 3 out of 5 is enough to decide).

I4. File server shared by 6 clients (mutual exclusion scheme).

Scheme	Messages per update (N = 6)	Failure risk
Central lock server	3	Lock server crash
Token ring	1–5 (+ continuous circulation)	Token loss
Ricart–Agrawala	2 × 5 = 10	Any client crash blocks everyone
Lamport	3 × 5 = 15	Same, plus needs FIFO
Recommended: central lock server (the file server itself can act as coordinator).		
- Fewest messages (3).		
- Simple and fair (FIFO queue).		
- The file server is already a single point of dependency, so the coordinator adds no new risk.		
- If it crashes, run the **Bully algorithm** to elect a new lock manager.		

I5. 7 servers; coordinator P7 crashes. Elect a new one with Bully and Ring.

Bully (P3 detects):

```
P3 —ELECTION→ P4, P5, P6, P7✗        P4, P5, P6 —OK→ P3
P4 —ELECTION→ P5, P6, P7✗            P5, P6 —OK→ P4
P5 —ELECTION→ P6, P7✗                P6 —OK→ P5
P6 —ELECTION→ P7✗ (timeout)
P6 —COORDINATOR→ P1, P2, P3, P4, P5  ⇒ P6 elected
```

Ring (P1→P2→…→P6→P1, P7 skipped; P3 starts):

```
P3: ELECTION(3) → P4 (replace 4) → P5 (replace 5) → P6 (replace 6) → P1 (fwd 6)
→ P2 (fwd 6) → P3 (fwd 6) → P4 → P5 → P6 receives own id ⇒ P6 elected
P6: ELECTED(6) → P1 → P2 → P3 → P4 → P5 → P6
```

Both elect P6, the highest alive process. Bully used 21 messages here (4+3+3+2+2+1+1 ELECTION/OK + 5 COORDINATOR); Ring used 9 ELECTION + 6 ELECTED = 15.